

One transaction. Every effect. Fully traceable.

A modular, multi-company ERP architecture centered on operational integrity, double-entry accounting and permanent user accountability.

PostgreSQL 17

NestJS 12 / TypeScript

React / Next.js 16

136-table schema baseline

Delivery boundary: all 28 design artifacts are mapped in this pack. The implemented release is a tested accounting foundation—not the complete production ERP. See the final chapter for exact coverage and remaining gates.

Rendered 30 September 2026 · KES functional currency first · Africa/Nairobi

Companion files: SQL migrations, full ERD, 583 foreign-key relationships, column dictionary, OpenAPI, source and test evidence.

Kenya Retail ERP — Architecture & implementation contract

Baseline 0.1, accompanied by increments 0.2–0.5 controls · 26 September 2026 · PostgreSQL / NestJS / React / TypeScript

Status: engineering blueprint, not a certification of production readiness. Database structures are a target baseline. Only capabilities listed as tested in STATUS.md are implemented. No live payment or fiscal integration is implied. Retail/wholesale, KES functional currency initially, online-only POS, perpetual weighted-average costing. Other currencies are designed but must remain disabled until settlement/revaluation tests pass.

Deliverable map

Requested deliverable	Location
1 Overall architecture; 2 Module architecture	§1–2
3 ERD; 4 Complete database schema	02-data-model.md, db/001-core.sql, db/002-domains.sql, additive migrations db/003-hardening.sql through db/006-company-and-warehouses.sql
5 Accounting; 6 COA; 7 Rule engine	§3–5
8 POS; 9 Purchasing; 10 Inventory; 11 Customer; 12 Supplier	§6–10
13 Payments; 14 M-Pesa; 15 Tax; 16 Cashier; 17 Expenses	§11–15
18 Approvals; 19 Attribution; 20 Audit; 21 State machines	§16–19
22 Permission matrix; 23 API	03-api-security.md
24 Sequence diagrams	§20
25 Reversals; 26 Reconciliation	§21–22
27 Testing; 28 Phases	04-delivery-quality.md

“Complete schema” here means the full target table baseline, not that every constraint or business service has been implemented. Domain-specific posting guards, fiscal mapping, reporting views and migrations will be refined and tested before those domains are released. The implementation status is deliberately separate from the design.

1. Overall architecture

Use a **modular monolith**, not independently committing microservices. PostgreSQL is the financial system of record. A command changes its document, ledgers, movements, tax facts, journal, audit and outbox within one database transaction. Async workers may deliver external side effects; they never independently invent ledger balances.

React workspace / POS

| HTTPS + secure authenticated session + CSRF

NestJS API

| Validation → Authentication → Membership → Permission → Scope

Command / TransactionCoordinator

├ WorkflowService / ApprovalService / NumberingService

├ PricingService / TaxService / InventoryService

├ CustomerLedgerService / SupplierLedgerService

├ PaymentService / CashierService

├ AccountingService (single posting gateway)

└ AuditService / OutboxService

| one connection, one transaction, company scope

PostgreSQL – relational documents + append-only ledgers + snapshots

├ outbox → workers → M-Pesa / eTIMS / email / object storage

└ read models → dashboards / reports / exports

Transactions receive an immutable context: authenticated actor, effective company, permitted branches, session, request ID, trusted IP, action, reason and idempotency key. Frontend actor fields are rejected, not ignored. No database writes use a client-supplied creator. UUID source registry with a real foreign key replaces unconstrained source_type/source_id polymorphism.

Deployment target: separate API and workers, static web, managed PostgreSQL with PITR, private object storage, TLS ingress, secret manager, telemetry and alerting. Redis can support throttling/work queues but is not financial authority. Containers run non-root. App DB role is neither owner nor superuser; migrations use separate credentials. Company RLS is defense in depth; authorization is still mandatory. Restore drills, load tests and independent security/accounting sign-off precede production.

2. Modules and ownership

Domain	Owns	May request effects from
Identity	users, sessions, role grants, security events	Audit
Organization	company, branch, warehouse, terminal	Identity, Audit
Core workflow	document registry, transitions, numbering, approvals	Audit
Catalog	products, UOM, categories, barcodes, price lists	Tax configuration
Inventory	movements, reservations, lots, counts, valuation	Accounting
Procurement	requisitions, PO, GRN, matching, supplier invoices	Inventory, AP, Tax, Accounting
Sales/POS	quotes, orders, invoices, sale lines, returns	Inventory, AR, Payments, Tax, Accounting
Partners	customers, suppliers, credit policy	Subledger
Payments	intents, confirmations, allocation, reconciliation	AR/AP, Cashier, Accounting
Cash management	shifts, floats, transfers, petty cash	Payments, Accounting
Expenses	requests, category/cost center, receipts	Tax, AP/Payments, Accounting
Accounting	accounts, rules, periods, journals, GL	Core/Audit
Tax/Fiscal	effective rules, tax facts, eTIMS submissions	Outbox, Accounting
Reporting	projections, reconciliations, exports	Read-only ledger contracts

Controllers invoke services. Shared services take a transaction connection rather than opening nested transactions. Domain modules cannot insert arbitrary journal lines outside AccountingService. Reports never mutate ledgers. Materialized caches are rebuildable and cannot be edited through an API.

3. Accounting architecture

Journal headers link exactly one source document and company, branch, currency, exchange-rate snapshot, accounting date, period and actor. Lines hold account, transaction debit/credit, functional debit/credit and dimensions (partner, warehouse, item, cost center as applicable). Both debit/credit totals must balance **exactly at currency scale**. A line has precisely one positive side, not both. Zero-value lines are omitted. Non-posting parent/inactive accounts are blocked.

A journal is constructed inside the database transaction, validated and sealed POSTED before commit. It must not be possible to commit an unsealed journal. Posted headers and lines are immutable at DB level. Reversal creates a new source and a new opposite journal; a relation marks the original as reversed without modifying its amount, author, date or lines. Draft manual journals are document payloads, not posted journal tables.

Closed-period guards hold a share lock on the selected period until commit. Close/reopen takes an exclusive row lock and therefore serializes with posting. Non-overlapping company periods use exclusion constraints. Period close requires reconciliation, pending posting review, accrual review and approval. LOCKED periods have no ordinary reopen command; exceptional administrative unlocking requires separate dual-controlled procedure.

Never allow standalone manual entries to AR, AP, inventory, cash-payment or tax control accounts. Authorized opening and correction services must create matching subledger/movement/tax/payment entries. The initial manual-journal API blocks all control accounts. An accountant can use non-control expense/equity accounts while operational services are unfinished.

Functional currency balances are authoritative for consolidated company books; original currency remains available. Foreign currency settlement uses historical carrying value and books FX differences to configured realized gain/loss. Period-end revaluation is reversible and explicitly linked. Intercompany transactions produce two separately balanced company journals under a linked workflow; never one journal spanning companies.

4. Configurable chart of accounts

Example starter structure (codes are configuration, not application constants):

Code	Name	Type	Control use
1000	Assets	ASSET	Parent, not postable
1100	Cash drawers	ASSET	Cash movement
1110	Petty cash	ASSET	Cash movement
1200	Bank	ASSET	Payment
1210	M-Pesa wallet	ASSET	Payment
1220	Card clearing	ASSET	Payment
1300	Trade receivables	ASSET	AR
1310	Supplier advances	ASSET	Supplier advances
1400	Merchandise inventory	ASSET	Inventory
1410	Inventory in transit	ASSET	Inventory
1500	Recoverable input VAT	ASSET	Tax
2000	Liabilities	LIABILITY	Parent
2100	Trade payables	LIABILITY	AP
2110	Goods received not invoiced	LIABILITY	GRNI
2120	Customer deposits	LIABILITY	Customer advances
2200	Output VAT	LIABILITY	Tax
3000	Equity	EQUITY	Parent
3100	Opening equity	EQUITY	Controlled migration
3200	Retained earnings	EQUITY	Year close
4000	Sales	REVENUE	Posting
4010	Sales returns	REVENUE	Contra revenue
5000	Cost of goods sold	EXPENSE	Posting
5100	Stock loss / damage	EXPENSE	Posting
5200	Purchase price variance	EXPENSE	Posting
6000	Operating expenses	EXPENSE	Parent
6100	Rent	EXPENSE	Posting
6200	Utilities	EXPENSE	Posting
6300	Cash over/short	EXPENSE	Approved variance
6400	Merchant fees	EXPENSE	Posting
6500	FX gain/loss	EXPENSE	Posting

Each company owns its own tree. Parent cycles are forbidden. Accounts used historically cannot be deleted or repurposed. Bank/wallet accounts are mapped to an account and currency; bank account details are protected from ordinary cashiers. Financial statement mappings are versioned, separate from account codes.

5. Rule engine

A versioned rule set maps symbolic legs to company-owned accounts. Select by transaction type and effective date, then optional branch, warehouse, product, category, payment method and tax category. Priority and specificity must resolve to exactly one rule; ambiguity or missing mapping blocks posting. Snapshot selected rule/version/account IDs with every journal. Never execute arbitrary rule code or eval expressions; allow-listed leg definitions select named totals computed by domain services.

Event	Debit	Credit
Cash sale	Tender accounts (net cash after change)	Revenue, output tax
Credit sale	AR	Revenue, output tax
Goods sold	COGS	Inventory at issued carrying cost
Customer receipt	Cash/bank/wallet	AR; deposits for unallocated advances
GRN	Inventory at provisional receipt cost	GRNI
Matched supplier invoice	GRNI, recoverable tax; inventory/variance if needed	AP
Direct non-stock invoice	Expense, recoverable tax	AP
Supplier payment	AP; supplier advances for excess	Cash/bank/wallet
Expense paid immediately	Expense, eligible input tax	Tender
Inventory shortage	Stock loss	Inventory
Inventory excess	Inventory	Approved stock-gain account
Internal cash transfer	Destination cash	Source cash

No double inventory capitalization: purchase invoice does not debit the entire Inventory a second time after a GRN. Match GRNI; split subsequent price differences between stock remaining and COGS/variance under an approved policy. GRNI remains reconciled by receipt line. Purchase-order approval itself creates no journal. Receipt taxation and invoice input VAT timing follow the configured tax policy, not blanket assumptions.

6. POS and sales workflow

1. Authenticated cashier opens/uses their own terminal session. One open session per terminal. Opening float is a posted transfer from safe, not unexplained cash creation.
2. Scan keyboard barcode, enter SKU or server-side search. Exact barcode match first. Unknown barcode shows controlled exception; never invent a price or product. F2 search, F4 customer, F6 tender, F8 hold, F9 submit, Escape close dialog. Per-user key mapping can override defaults.
3. Cart quotes use server pricing/tax and stock visibility; they do not post. Walk-in requires no permanent party and cannot receive credit. Holds persist cart version and expire reservations; they are not sales.
4. Server revalidates catalog, price policy, discount approval, tax date, active warehouse, session ownership, reservation, customer credit and tender facts.
5. Cash amount tendered is distinct from applied amount and change. Mixed tenders must sum to invoice total; non-cash overpayment is an advance, never automatic change. Card requires verified provider confirmation or a controlled manual settlement workflow.
6. Lock stock valuation keys in deterministic company/warehouse/product/lot order. Lock customer credit exposure and allocations. Check available stock/credit including active reservations.
7. In one transaction create source/sale/lines, applied payments, movement/cost snapshots, AR if credit, tax facts, cash movements, balanced journal(s), audit and outbox. Post once, then return receipt.
8. Print via local browser/receipt bridge. Reprint marks COPY and audits. Transport/printing failure does not undo a committed sale; retry retrieves the existing idempotent result.

Receipt shows company/branch, number, date, cashier, terminal, item quantities/prices/discounts/tax, total, tender, change and fiscal status. Fiscal reference/QR appears only when real fiscal confirmation is obtained. UI must not label pending fiscal output as a compliant tax invoice.

7. Purchasing workflow

Requisition → approved PO → partial/full GRN → matched supplier invoice → AP → allocated payment. Ordered/received/invoiced/returned quantities derive from line links, not manually editable header booleans. Overage requires authorization. Rejected/damaged items enter quarantine when physically accepted, otherwise do not inflate sellable stock.

Three-way match compares supplier, SKU/UOM, received-minus-already-invoiced quantity, unit cost, tax basis/rate, currency and invoice uniqueness. Company+supplier+normalized supplier invoice number is unique. Quantity/price/tax tolerances are versioned; over-tolerance moves to exception approval, never silently writes off a discrepancy. GRN locks PO lines. Invoice locks receipt allocations. Backorders remain visible. Landed costs create cost-adjustment movements and journals, never mutate historical movements.

Supplier returns reference receipt and cost allocation. Block quantities greater than accepted less previous returns. Physical dispatch can create a supplier claim/clearing balance pending supplier credit note; AP/input-tax effects depend on the accepted fiscal credit document. Do not imply supplier acceptance merely because stock was sent back.

8. Inventory and valuation workflow

Authoritative stock = sum signed movements by company, warehouse, product, lot/serial and ownership/status. A balance/valuation projection can be locked for performance; no UI CRUD for it. Movement includes source line, quantity, functional value, actor and original movement for reversals. Zero-quantity value adjustments are permitted only via a specialized valuation command, never ordinary stock receipt/issue.

Weighted average per company/warehouse/product valuation pool: receipt adds quantity and carrying value; issue cost is current value / quantity at high precision, rounded to base currency at posting; issuing the final quantity consumes the entire remaining value. Transfers preserve carrying cost; not selling price. At zero quantity, carrying value must be exactly zero. Stock reservations affect available quantity but not on-hand or GL. Negative stock is disabled initially. Backdated cost-impacting commands are blocked before the last valuation event; do not silently recalculate previous COGS. FIFO is a future cost-layer implementation, not a selectable non-working option.

Transfer request → approval → dispatch to transit → partial receipts → discrepancy decision → completion. Transit is a real location and GL dimension. No goods disappear between branches. Cross-company movement is intercompany procurement/sale, not an internal transfer. Counts freeze scope or record cut-off movements; independent blind count and approved variance precede stock and GL adjustments. Serial uniqueness, batch expiry, quarantine eligibility and sale UOM conversions are enforced at posting. Expired/quarantined stock cannot be sold through ordinary POS.

9. Customer workflow

Create identifiable party and account; credit permission controls terms/limit changes with audit. Credit exposure = open receivables + configured commitments/reservations, adjusted for eligible credits. Lock account before checking new credit. Terms and due dates snapshot on invoices. Opening balances enter through approved migration documents with AR reconciliation, never a customer.balance field.

Invoices debit customer subledger/AR; receipts and credit notes credit it. Advances map to customer-deposit liability and are separately presented until applied. Allocations are immutable events linking receipt/credit to invoice with reversal records. Lock both open items before allocation; do not over-allocate, cross-company allocate or silently mix currencies. Aging uses unallocated residual amounts and due-date buckets, not net customer balance. Statements include original and reversing events with running balance.

10. Supplier workflow

Vendor onboarding and bank-detail changes need stronger permissions/approval. Supplier invoices credit AP; payments and recognized credit notes debit AP. Supplier advances are asset open items until allocated. Supplier statement liability balance is credits minus debits. Invoice duplicate checking is independent of idempotency. Payment approval is distinct from invoice approval and bank release. Payment/export initiator must not approve their own disbursement where policy requires independent users.

11. Central payments workflow

Separate **intent**, **provider attempt**, **verified receipt/disbursement**, **allocation**, **settlement** and **accounting reversal**. Tender methods select company-owned bank/wallet/cash/clearing accounts. Each payment retains initiator and verifier/poster origin, amount/currency, provider references and source. Ledger records reflect actual movement, not merely requested movement.

States: PENDING → SUCCESSFUL | FAILED | CANCELLED | TIMEOUT; SUCCESSFUL → REVERSED only after actual confirmed reversal and reversing entries. TIMEOUT is unresolved until queried; a late success is retained and must not be lost. UI cancellation does not cancel external funds. Card charges settle through clearing; merchant fees are separate expense entries, not unexplained netting. Settlement allocation matches provider statement rows with tolerance exceptions requiring approval.

12. M-Pesa workflow and external atomicity

In local transaction create payment intent and outbox, reserve cart stock and credit if required; commit before contacting provider. Worker initiates STK with retry-safe correlation and records request IDs. Ambiguous network timeout is queried/reconciled before retrying initiation. Raw callback is untrusted input: size/type validation, expected request lookup, expected merchant identity, amount/currency/phone reference checks where available, durable inbox and unique provider receipt key. **Do not assume an HMAC signature exists in a provider interface**. Select verification mechanisms from the actual contracted API; source IP controls are only defense in depth. Independently verify payment through available authenticated provider query/statement capability before financial success. If evidence is insufficient, hold for review rather than posting.

Verified result triggers a local atomic posting command attributed to integration service principal and original human initiator. Duplicate callback or retry returns previous outcome. If stock reservation expired or period closed, recognize verified funds as customer/unidentified receipt liability with explicit reconciliation exception; do not fabricate a sale or lose received money. Verified provider funds cannot be rolled back by rolling back the local database. Refund is a new external operation and local reversing/clearing process. Outbox/inbox processing is at least once with idempotent consumers, not a claim of global exactly-once delivery.

No live adapter is delivered in Phase 1. Safaricom portal access, shortcode ownership, sandbox fixtures and production approval must be supplied/verified before release. The attempted public Express query documentation URL did not expose a usable specification; protocol details remain a gate, not invented implementation.

13. Tax architecture and Kenya boundary

Central engine selects effective version from item category, partner eligibility, transaction type, location and tax point. Preserve taxable/zero-rated/exempt/non-taxable as distinct categories even when amount is zero. Store rate, version, basis, discount allocation, inclusive flag, rounding method, input eligibility, tax-point date, currency/base amounts and mapped account. Multiple taxes need defined ordering/compounding. Configuration changes create future versions; historical snapshots never change.

At the time of design, KRA's VAT page lists a 16% general rate and 0% zero-rate, distinguishes exempt supplies, and says the former petroleum 8% rate was deleted from 1 July 2023. Do not infer an item's legal category from its name. The same page describes tax point as the earliest applicable event, including receipt of part/full payment. Therefore advance-payment VAT must be modeled and cleared on final invoicing without double charging. Input VAT requires eligibility/supporting evidence; nonrecoverable tax is capitalized/expensed as appropriate. **No live tax rates are activated automatically by the foundation seed.**

Primary reference retrieved 26 September 2026: <https://www.kra.go.ke/individual/filing-paying/types-of-taxes/value-added-tax> . eTIMS entry point: <https://www.kra.go.ke/online-services/etims> . Obtain current legislation, KRA integration specification and accountant approval before activation. This design is not tax advice or eTIMS certification.

Tax rounds at documented line/currency scale, then sums. Inclusive net is $\text{gross}/(1+\text{rate})$, rounded per configured policy; $\text{tax} = \text{gross} - \text{net}$. Allocation of header discount uses largest-remainder distribution with stable line ordering. Rounding differences only use an explicit configured rounding leg within a documented tolerance; unexplained residuals fail posting.

Fiscal outbox stores immutable invoice snapshot/hash, reference, attempts, fiscal receipt/QR and failure status. Accounting date and fiscal issuance state are separate. Adapter-specific legal issuance rules can block final receipt or queue under an approved outage procedure. Retry cannot create a second fiscal invoice. Credit note references original fiscal invoice. Readiness includes eTIMS sandbox/approval, legal tax mapping and outage procedure.

14. Cashier and petty cash workflow

OPEN → ACTIVE → RECONCILIATION → CLOSED. Cashier owns terminal session. Expected cash derives from signed drawer movements: float + cash sales + collections + cash transfers in – cash refunds – paid-outs – transfers out. Credit/M-Pesa/card sales do not inflate cash expected. Physical count is blind and captured separately. Variance = actual – expected. Supervisor reviews reason/evidence, posts over/short and transfers counted cash to safe; never overwrites original movements. Late verified noncash payments remain linked to originating session with explicit late-settlement event, not forced into another cashier's cash.

Petty cash follows the same movement ledger with custodian/cash-account dimension. Replenishment is bank → petty cash; approved expense reduces petty cash. A counted shortage needs a variance transaction. Count, reconciler and approver are separate attribution fields.

15. Expense workflow

DRAFT → PENDING_APPROVAL → APPROVED → POSTED. Lines have category, expense account, cost center, net/tax/gross, eligibility and supplier/payee. Attachments go to quarantine before virus scanning; link object/hash/size/MIME/uploader and source. Approved revision invalidates previous approvals. Pay later creates AP; paid now uses payment engine in same posting transaction. Foreign supplier/WHT/other taxes require separately configured rules; unavailable rule blocks the relevant workflow. Reversal retains attachment and reason, and creates counter-entry/credit document rather than deleting expense.

16. Approval workflow

Versioned policies scoped to company/branch/event define currency, threshold, required steps, role/permission, creator-versus-approver and creator/approver-versus-poster separation. Threshold comparison uses documented functional amount, never a user-manipulable currency. Record policy version, decision, actor, timestamp, comments and document revision. Value/lines/currency changes invalidate pending approvals. A rejection records decision; revising creates new approval cycle. Delegations are time-bound, scoped and audited. No role name is hard-coded in service logic.

At least two-person approval is required for production configuration, opening balances, account/control mapping, period reopening, bank details, refunds, stock adjustment and high-value supplier payment. Three-person rules can be enabled. Initial foundation journal workflow implements independent creator/approver/poster when configured in its company policy; full multi-stage thresholds remain pending.

17. Attribution architecture

users includes non-login service principals with type HUMAN/SYSTEM/INTEGRATION and service code, so every creator/poster foreign key is non-null. HUMAN accounts authenticate; service principals are restricted to assigned machine workflows and cannot log into the UI. An `initiated_by` link preserves the initiating human. `Created_by` never changes. Each transition writes an append-only event with actor; header approval/posting metadata is derived or write-once. Database guards validate actor context for protected source/audit/journal inserts. Cross-company foreign keys include company ID. Actors must have active membership when commands execute; revocation is checked server-side on each request.

18. Audit architecture

Append-only audit records contain event ID, UTC timestamp, effective company, actor/type/service, initiating user, action, module/entity/source, request/session, trusted IP/device, redacted old/new values and reason. Do not copy secrets/tokens/passwords/provider credentials into audit. Record draft edits, approval, posting, reversal, permission change, period transition, security events, exports and receipt copies. Failed command/security audit uses an independent committed record because transaction rollback otherwise removes it.

Normal users and runtime DB roles cannot update/delete audit. Operational FK deletion is RESTRICT, not CASCADE. A privileged DBA can still tamper with a database; production must export signed audit batches to restricted immutable/WORM storage and monitor gaps. A local hash chain alone is not proof

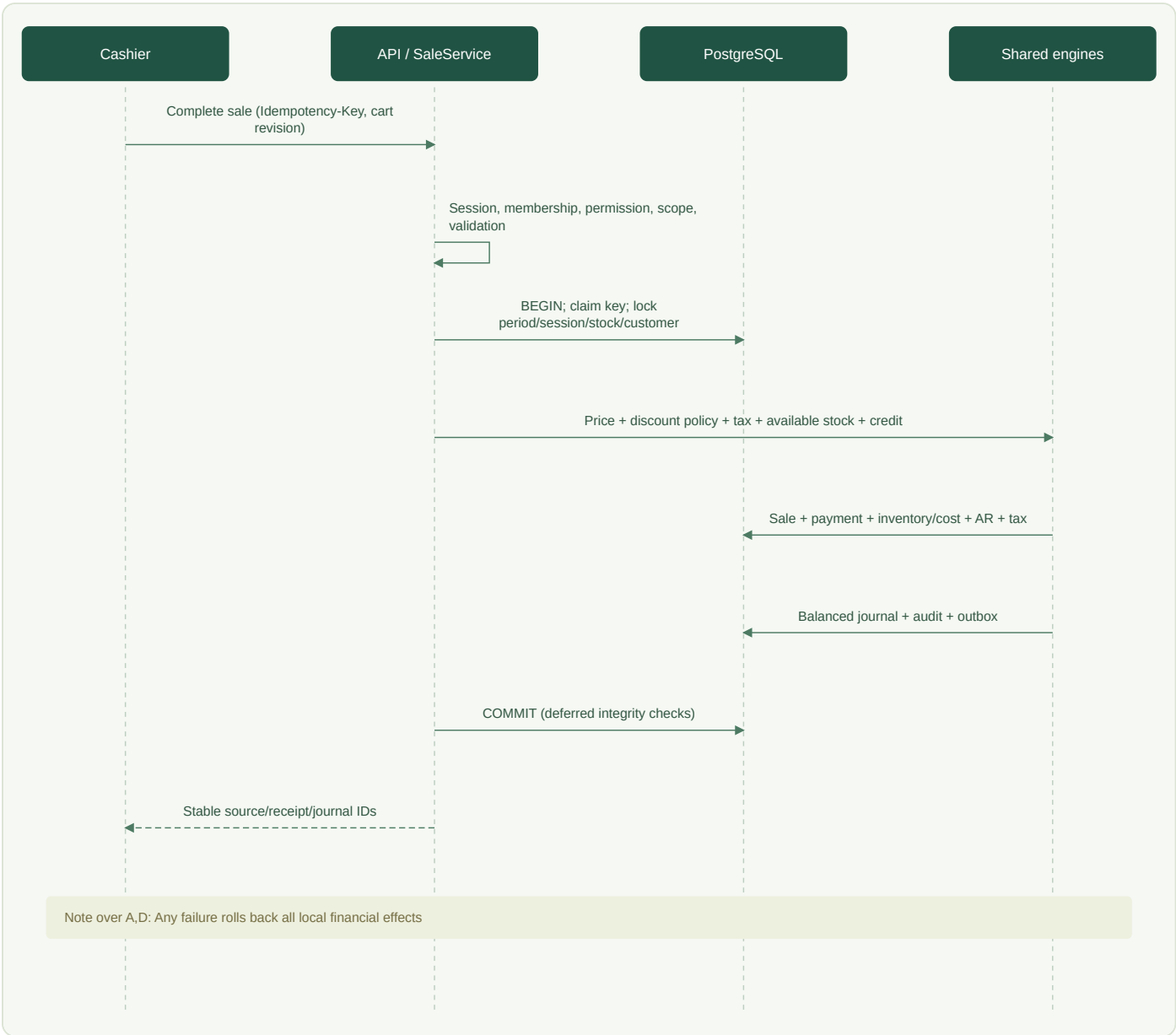
against administrators. Retention/legal holds and Kenyan privacy/ODPC obligations need owner approval; don't invent a blanket purge duration. PII access/export is permissioned and logged.

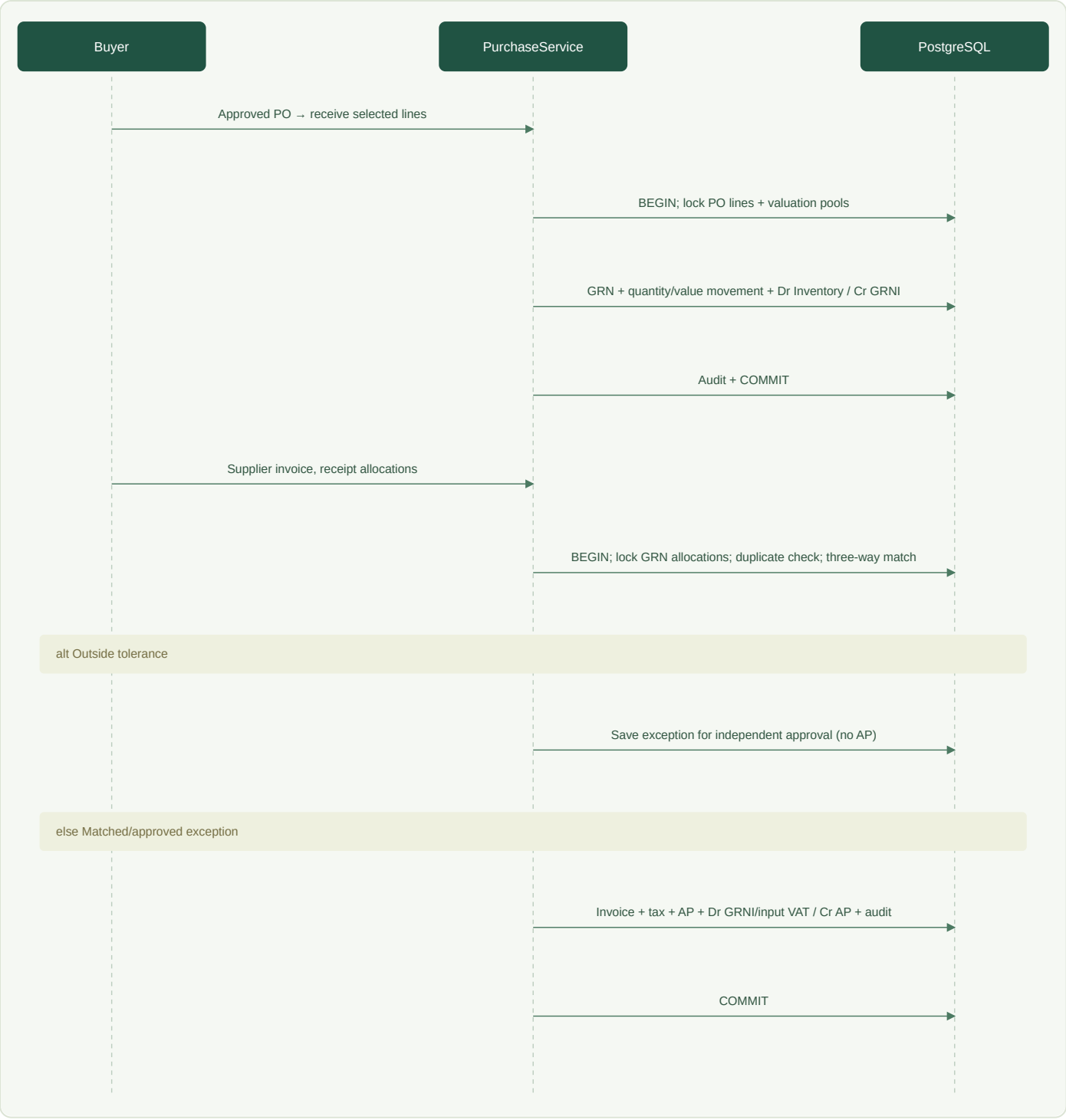
19. State machines

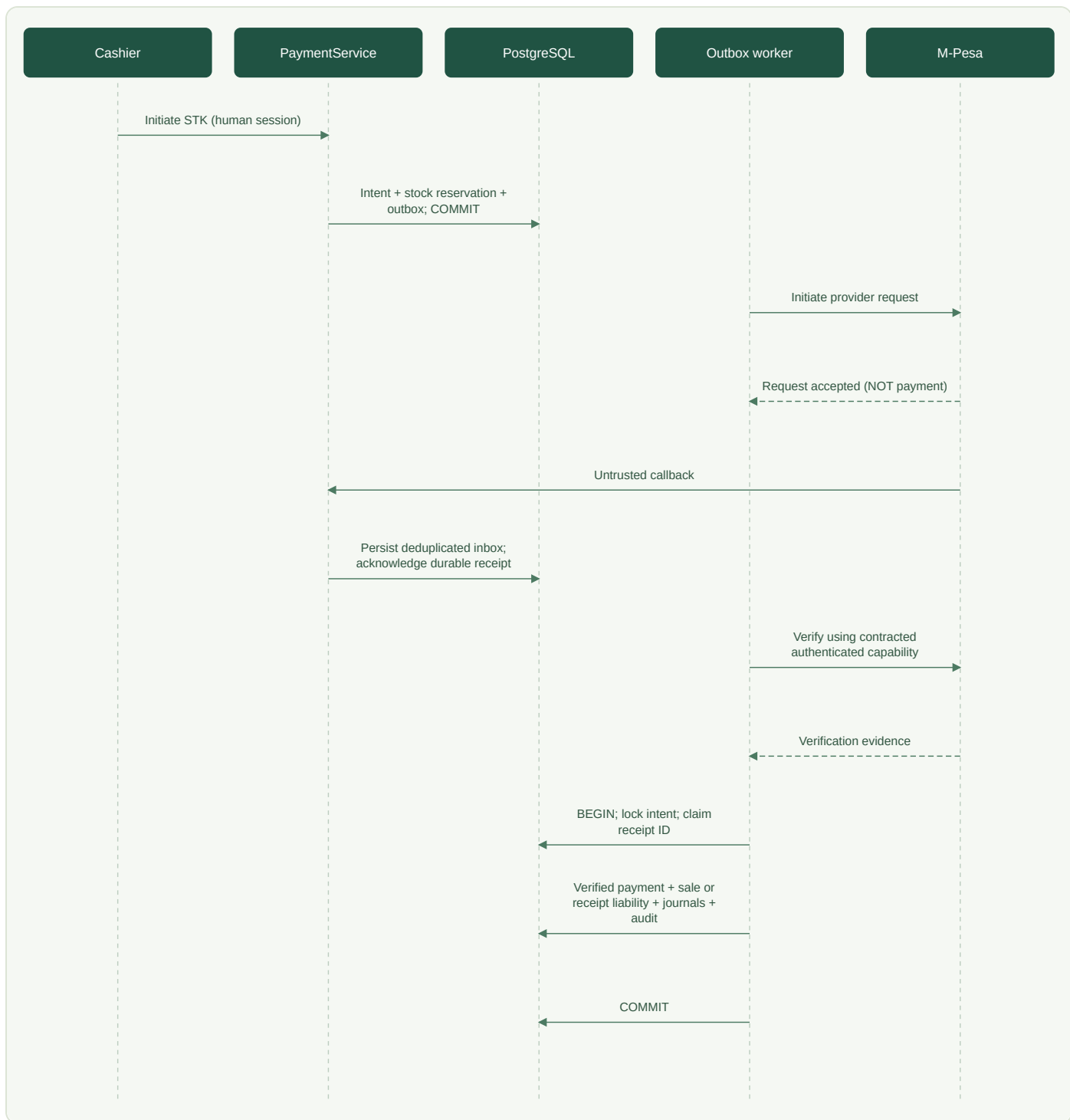
Document	Allowed transitions	Financial effect
Quote/order/PO	DRAFT → PENDING_APPROVAL → APPROVED → PARTIAL → COMPLETED	Commitments only until fulfillment
Draft invoice/expense/manual journal	DRAFT → PENDING_APPROVAL → APPROVED → POSTED	Atomic posting only at final transition
Rejected draft	PENDING_APPROVAL → REJECTED → DRAFT (new revision)	None
Cancellation	DRAFT/PENDING_APPROVAL/APPROVED → CANCELLED, if no posted children	None; posted children require compensations
Posted source	POSTED + linked compensating source	Original immutable; effective display REVERSED or PARTIALLY_RETURNED
Stock transfer	DRAFT → APPROVED → DISPATCHED → IN_TRANSIT → PARTIAL → COMPLETED	Paired movements, preserving value
Count	DRAFT → COUNTING → SUBMITTED → APPROVED → POSTED	Approved variance only
Period	OPEN → CLOSED → LOCKED; CLOSED → OPEN authorized	Lock/audit, not arbitrary status patch
Payment	PENDING → SUCCESSFUL/FAILED/CANCELLED/TIMEOUT; verified late resolution	Only verified funds create money movement
Cashier	OPEN → ACTIVE → RECONCILIATION → CLOSED	Movements retained

Transitions are domain commands with preconditions and version checks. There is no general PATCH status endpoint.

20. Transaction sequences







21. Reversal architecture

Original posted source is immutable; document_links and immutable reversal source establish reversal relationship. Full reversal allowed only once by uniqueness; partial returns track quantities/allocation against original lines and cumulative return limits. Reversal requires date in open period, independent approval as configured, permission and reason. Reverse source and journal preserve original tax and cost snapshots, not today's tax or selling cost. Sales return restocks at original issued value where physically returned and saleable; damaged returns enter quarantine and separate impairment if appropriate.

An exchange = return + new sale with explicitly allocated credit. Cash refund is separate payment movement tied to return. Allocated payment reversal first reverses allocations to reopen invoices; cannot leave paid invoices with cancelled funds. Reversing a GRN already invoiced/sold requires dependency-aware compensations, not blind negative stock. A supplier invoice already paid cannot be cancelled as if payment vanished. Posted manual journals may be reversed by an independently approved reversal request; no direct header/line edit.

22. Reconciliation architecture

Run on-demand and scheduled, same-company functional currency, consistent repeatable-read snapshot and cut-off. Store run ID, ledger watermarks, filters, rule version, expected, actual, difference and offending source IDs. No balancing plug. Alert owners; material unexplained differences block period close.

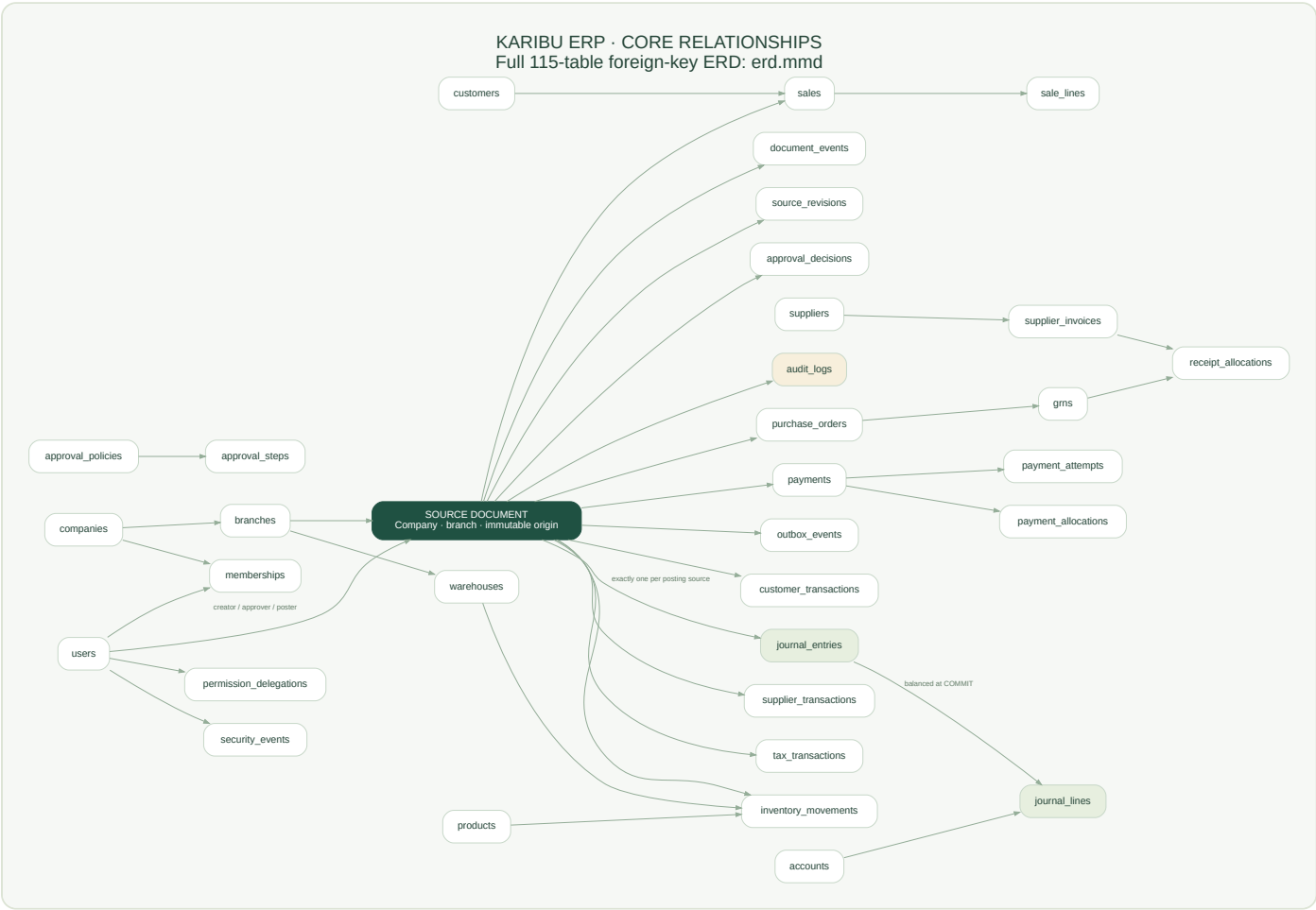
Check	Authoritative comparison
AR	customer debit-credit subledger by control account = AR GL debit-credit
AP	supplier credit-debit subledger = AP GL credit-debit
Deposits/advances	respective open-item ledger = liability/asset GL
Inventory	signed movement value by valuation pool = Inventory GL; qty projection = signed movement qty
GRNI	uninvoiced receipt carrying values = GRNI liability
Tax	input/output tax signed facts separately = mapped tax GL
Payments	posted money-movement debit-credit by mapped account = payment-control GL, including fees/transfers/openings
Drawer	signed session cash movements = expected drawer cash; physical count separate
GL	every journal balanced and global trial balance debit-credit = 0
Allocations	allocated totals ≤ settled receipt and open invoice amounts; reversals unwind exactly
Integration	provider settlement total = locally verified receipts/fees/refunds + explicit outstanding items

Each control account has a permitted posting-origin set. Empty domains do not count as tested reconciliation: until their posting services are implemented, UI reports “not implemented”, not “all reconciled”. Report reads original and reversal journals, not only unreversed records.

Data model and integrity boundaries

Full executable target DDL is split into 001-core.sql and 002-domains.sql. 003-hardening.sql adds core guards/runtime grants. erd.mmd is generated from all actual foreign keys, not a selective drawing. data-dictionary.csv is generated from PostgreSQL columns. Neither table presence nor an FK means the domain is functionally released.

Key relationships



Source-document supertype

Source carries common company/branch/date/currency, creator and state metadata. Domain tables carry specialized facts and a **unique composite FK** to source. Child lines derive creator of the document via the source, with row insertion actor also retained. Source ID is the referential source of all effects; operational type is stored on registry. Using a registry prevents orphan journal polymorphic IDs and permits full forward/reverse navigation. Historical source number and creator are immutable. Generic manual-journal drafts store a proposed line payload; posted lines are normalized with account FKs and checks.

Header states do not erase historical facts: reversal relation creates effective display status while original POSTED source remains untouched. Return line original-sale/receipt FKs preserve actual tax/cost origin. Role/permission assignments are relational configurable data. Company-role membership and all domain relationships use composite tenant keys.

Authoritative and derived data

- Authoritative: posted journal lines, movement values/quantities, customer/supplier events, tax facts, payment/cash events, source/approval/audit history.
- Derived: customer/supplier balances, cash/bank balance, control totals, outstanding invoice amount, quantity remaining on order/receipt, inventory projection and shift expected cash.
- `inventory_balances` is a lockable rebuildable projection, not a public editing endpoint. Phase 1 runtime receives no domain write privileges, including this table.
- Draft totals are provisional; posting recalculates from trusted snapshots/rules. Historic amounts cannot be recalculated using today's rates.

Precision and time

This first executable baseline deliberately fixes ledger monetary scale at **numeric(20,2)**, supported initial currencies KES/USD/EUR each scale 2; foundation API enables functional-currency journals only. Quantity/unit price use `numeric(20,6)`, tax/FX rate `numeric(20,10)`. Currency scale up to 4 is modeled, but currencies requiring another ledger scale must not be activated until a migration and full arithmetic suite are delivered. Use `Decimal.js` at precision 40 in services and decimal strings in transport. Never JavaScript arithmetic for ledger totals. UTC `timestampz` for events, date for company-local business/tax/period date. Rate and cost snapshots are immutable.

Constraint coverage

Core: FK/NOT NULL, unique sequence scope and document number, non-overlapping periods, source actor/state/origin guards, independent approval/posting, source/journal consistency, account company, currency/base checks, one-sided positive lines, deferred aggregate journal balance at commit, immutable journal/line/audit/event, source requires posted journal, idempotent source posting. Company RLS guards runtime reads/writes, with application membership/permission checks.

Domain baseline: all target tables, composite company foreign keys, positive quantity/payment checks, unique supplier invoice reference and provider receipt, line arithmetic, append-only movement/subledger/approval/cash/tax/allocation records, exact barcode uniqueness, one open terminal session, effective tax-range exclusions. **Pending before release:** domain cross-row consistency triggers (product/lot pairing, same-branch terminal/store pairing, header/line sums), domain-specific approval/revision policies (core journal policies are implemented), serial/expiry/quantity allocation invariants, movement/journal reconciliation guards, control-account-origin enforcement for domain posting, immutable configuration versions, currency settlement rules and full source/domain-type matching. Phase 1 runtime cannot write these domain tables; no claim that constraints alone finish the ERP.

Index and search policy

Every target table has company/time index; FKs/search-heavy joins get additional indexes during domain implementation and query-plan testing. Barcode/SKU/product code and partner numbers are exact unique lookup keys. Names use `pg_trgm`. Audit/source history has company+actor+date access paths. Large report scans use date/company filters and cursor pagination; dynamic sort identifiers must be allow-listed. No unbounded lists for front-end selectors.

Migration policy

Migrations execute under separate owner role and are ordered/checksummed by deployment tooling. Runtime is non-owner with RLS enabled and explicit grants. No implicit CASCADE delete of historical transactions. Additive schema changes first, backfill with attributed system documents where financial, then validate constraints, then enforce. Rebuild projections without changing authority. Seed is **development only** and creates sample companies/users/COA/open periods, not unexplained opening balances.

Additive control schema, increment 0.2

Migration 004 adds permission delegations, append-only security events, atomic authentication rate buckets, encrypted authentication delivery outbox and immutable source revision archives; it extends existing memberships, roles, policy/step, decision and source tables without replacing posted history. Current exported application schema: **116 tables, 1,212 columns, 490 foreign keys**. `schema_migrations` is separate operational metadata and excluded from these application counts. See control contract.

Migration 005 adds version/creation/update metadata to branches, immutable identity checks, dependency-guarded deactivation and deferred runtime audit enforcement. No application tables were added; branch-created timestamps for legacy rows remain unknown rather than backdated.

Migration 006 adds company and warehouse versions/edit attribution, immutable identity/hierarchy guards and deferred runtime audit checks. A non-public fixed-search-path SECURITY DEFINER helper checks all current operational warehouse references without exposing unreleased tables to the runtime. Legacy warehouse creation timestamps stay NULL. Basic configuration is implemented, not inventory operations or safe operational closure.

Migration 007 adds `account_change_requests` and account version/edit/approval-link metadata. Composite company FKs bind proposals, parents, targets and applied accounts. Deferred constraints require account application and exact audit snapshots in the same transaction. Runtime writes cannot bypass independent approval. Legacy account-created timestamps stay NULL. See COA workflow.

Reviewed initial product registration (017)

Adds normalized product requests/proposed barcode and tax children plus dated tax profiles, and guards existing product/barcode/tax publication. Native SQL snapshots preserve 20-digit decimal prices; exact-string authorized API projections do not mutate stored audits. Four new tables produce the 132-table/1397-column/560-FK export. Independent immutable review, whole-interval reviewed tax coverage, publication identity and deferred complete audits are enforced. No stock runtime writes or migration 018 exist yet. See `inventory/product-master-next.md` and `inventory/weighted-average-contract.md`.

Released 018 stock integrity

`inventory_adjustment_lines` and `inventory_valuations` bind immutable source proposals and revisioned valuation snapshots. Existing inventory movements now reference valuation and pool version; projections retain latest valued date. Native checks couple reviewed rule, original reversal source/movement/journal, movement, projection, exact audit and balanced GL. All 001–018 migrations are immutable. Current export: 134 tables, 1426 columns, 571 foreign keys. Table presence outside released workflows is not functional completion.

Released 019 customer masters

Initial independently reviewed registration uses `customer_change_requests`, existing customers and `customer_accounts`. Exact proposer/reviewer/policy metadata and native audits are immutable; deferred final-row guards reject post-BEFORE-trigger publication corruption. No `customer_transactions` row or balance is created. Current export: 135 tables, 1453 columns, 579 foreign keys; 001–019 applied and immutable.

API, authorization and user experience contracts

Contract conventions

Target API `/api/v1`; money/quantity/rates are decimal **strings**, never JSON floating point. UUID IDs; ISO UTC timestamps; ISO local business dates use Africa/Nairobi unless company overrides. Company selected by X-Company-ID but accepted only after membership validation. Branch/warehouse/terminal scope is checked independently. Pagination defaults 25, maximum 100, stable allow-listed sorting and cursor (or bounded offset in foundation); q length bounded and SQL values parameterized. Search indexes cover normalized identifiers, phone, PIN, barcode and trigram name. All responses avoid secrets and unnecessary PII.

Write commands require Idempotency-Key (bounded opaque identifier) and optimistic revision where editable. Scope idempotency to company+actor+command+key; canonical body hash must match on replay, otherwise 409. A completed command returns its original source identifier. Provider deduplication is additionally unique by company/provider merchant/external transaction ID, independent of human request keys. Accounting posting is unique by source/event. No duplicate posting on retry.

Errors: 400 malformed/invalid; 401 no valid session; 403 denied permission/scope; 404 not visible/found; 409 state/version/idempotency conflict; 422 valid input failing business policy; 429 rate limit; 503 external/temporary unavailable. Error object: {code,message,requestId,details?}. Validation responses expose safe field-level details; no SQL, stack, token or cross-company information. Financial failures are never represented as 200 success. 202 is only accepted-for-processing and must include status URL.

Foundation API (see implementation status for exact coverage)

Method / path	Permission	Purpose
POST /auth/login	public, throttled	Email/password → opaque session + CSRF token
POST /auth/logout	authenticated + CSRF	Revoke current session
GET /auth/me	authenticated	User, companies, permissions; no token hashes
GET /accounts	accounting:view	Company chart
GET /periods	accounting:view	Company periods
POST /periods/:id/close	periods:close	Lock/check/close with audit
POST /periods/:id/reopen	periods:reopen	Reasoned reopening, audited
POST /journals/drafts	journals:create	Create source document + immutable creator
POST /documents/:id/submit	journals:create	Creator submits draft
POST /documents/:id/approve	journals:approve	Independent approval
POST /documents/:id/post	journals:post	Post balanced non-control manual journal
POST /journals/:id/reversal-drafts	journals:reverse	Create request; approval/post required
GET /documents	journals:view	Paginated/searchable sources
GET /documents/:id	journals:view	Attribution, transitions, journal drilldown
GET /journals	accounting:view	Immutable posted journals
GET /reports/trial-balance	accounting:view	Derived account debit/credit totals
GET /reports/integrity	accounting:view	Core journal/source invariants only
GET /audit	audit:view	Company-filtered events

Target domain command catalog (not implemented by merely declaring routes)

Domain	Read/search resources	Commands
Identity	users, roles, permissions, login-history, sessions	activate/deactivate, grant/revoke, reset request/consume, revoke session
Organization	companies, branches, warehouses, terminals	configure, deactivate; audited
Catalog	products, categories, prices, barcodes, tax-categories	create, revise, activate/deactivate
Sales/POS	carts, holds, quotations, orders, sales, receipts	quote, hold, resume, reserve, checkout, release, cancel draft, reprint
Customers	customers, accounts, statements, aging, open-items	onboard, change-credit-policy, credit-hold, migrate-opening
Suppliers	suppliers, accounts, statements, aging, invoices	onboard, change-bank-details, migrate-opening
Purchasing	requests, orders, receipts, invoices, match-exceptions	submit/approve/reject, receive, match, approve-variance, post
Returns	sale-returns, purchase-returns, credit/debit-notes	request, approve, receive/dispatch, post-credit, refund
Inventory	on-hand, movements, valuation, reservations, lots	reserve/release, dispatch/receive-transfer, start/count/approve/post-count, adjust-value
Payments	intents, attempts, allocations, settlement	initiate, verify, allocate/unallocate, approve-disbursement, reverse, reconcile
Cashiers	sessions, cash movements, cash counts	open, activate, record-tender, reconcile, approve-variance, close
Expenses	expenses, categories, petty-cash	submit/approve/post, replenish, count, reverse
Tax	rules, versions, facts, fiscal-submissions	schedule-version, validate-configuration, submit-fiscal, retry, credit-note
Accounting	accounts, rules, periods, journals, GL	configure/version, draft, submit/approve/post, reverse, close/reopen
Reporting	sales, purchases, cash, inventory, GL, P&L, BS, cash flow, aging	request export, poll job, retrieve signed download
Audit/attachments	events, file metadata	upload-ticket, scan, link, retrieve-authorized; no posted delete

Nested effects never receive their own public “set balance” routes. Unimplemented APIs must return 404/501 rather than mocked success.

Permission matrix (starter grants, editable data)

C=create, V=view, E=edit draft, A=approve, P=post, R=reverse/return. These are descriptions; stored keys remain granular action permissions. Roles are data, not special cases in controllers.

Domain	Cashier	Storekeeper	Purchasing	Accountant	Manager	Auditor
POS/sales	CV, scoped checkout	V	—	V	V/A discounts	V
Customers	V/search, C permitted	—	—	CV, credit proposal	A credit	V
Refunds/returns	C request	Receive authorized	C supplier return	P approved	A	V
Stock	V assigned store	CV counts/transfers	V	V valuation	A adjustments	V
Purchasing	—	V/receive approved	CVE/submit	V match/P invoice	A PO/variance	V
Payments	Receive assigned shift	—	Propose supplier	CV/P approved	A disbursements	V
Expenses	Request only	Request only	Request only	CV/P	A	V
Journals/periods	—	—	—	CVE/P, close proposal	A/reopen separately	V
Reports	Own session	Assigned stores	Purchasing	Finance	Company	Company V/export
Identity/configuration	—	—	—	V finance config	—	V changes
Audit	Own receipt activity	Own stock activity	Own purchasing	Finance V	Company V	Company V

Security administrator manages users/roles but has no automatic financial-posting powers. Break-glass is time-bound, independently approved and prominently audited. Role assignment cannot escalate beyond assigner's delegated permission set. Branch scope constrains grants; explicit company membership is required. Segregation rules apply even if a user has every permission.

Authentication and security target

Argon2id password hashing, unique normalized email, breached/common password checks, password reset single-use hash with expiry, generic reset response, revoke all sessions after reset/deactivation. Opaque random session token, hash stored in DB, absolute and idle timeout, server-side revocation; HttpOnly Secure SameSite cookies in production with CSRF for writes. Session fixation prevented by new random token at login. Successful login history and failed attempts recorded without plaintext secrets. Rate limits keyed by normalized account and trusted IP; bounded lockout avoids indefinite denial of service. MFA/step-up required before high-risk production actions (future release gate).

Machine principals use managed credentials/workload identity, scoped permissions and short-lived access; they are not anonymous users. Tenant context is transaction-local and not reused across pool checkout. RLS app role cannot bypass policies. Composite foreign keys prevent attaching another company's account/customer/warehouse even when a caller knows its UUID. Tests must attack cross-tenant reads and writes.

Uploads: allow-listed types, magic-byte checks, bounded size, random object names, malware quarantine, private bucket, short-lived authorized downloads, content disposition, no user executable SVG/HTML inline. No frontend storage of payment secrets; card integration uses provider tokenization and never stores PAN/CVV. Strict CSP, safe headers, allow-listed CORS, TLS and no public DB. Dependency scanning/SBOM, secret rotation and signed release artifacts.

UI/UX target

Workspace with company/branch switcher, persistent user/session identity, dashboard and permission-filtered nav. Large searchable inputs with debounced server query, cursor pagination, keyboard-selectable results; no thousand-item dropdown. Tables support filter chips, stable sort, explicit timezone, current scope, user and source drilldown. Loading skeleton, empty explanation, recoverable error and request ID. Show money in currency, never abbreviate exact document totals.

Document view includes workflow banner, timeline of creator/approver/poster/reverser, line totals, operational links, tax snapshot, payment state and balanced-journal panel. Display action permissions and reason for disabled action; backend repeats validation. Dirty-form warnings, explicit destructive/financial confirmation, idempotent retry after network failure. Light/dark themes, WCAG keyboard/focus/contrast, desktop/tablet layouts. POS optimizes scan focus and cashier speed while preventing Enter from accidentally re-submitting a posting.

Offline checkout is disabled initially. Future offline mode requires device identity, signed queued commands, durable local encrypted queue, temporary IDs and deterministic idempotency. Offline displays provisional receipt only; server validates pricing/stock/credit/fiscal policy and may quarantine conflicts. It must not claim committed stock or payment before synchronization.

Increment 0.2 implemented additions

Member and delegated role administration, independently published threshold/quorum policies, company security events, own-session lifecycle, encrypted password recovery/change, and reasoned source revision/rejection/cancellation are implemented. At increment 0.2 the generated OpenAPI documented 36 paths. Access edits share the command transaction/idempotency boundary with accounting but require exclusive company authorization locks. Public recovery never returns reset tokens; login requires application/json. See the preimplementation control contract and verification checkpoint and current status. Read payloads still use generic Record schemas in parts of the reference specification; this is not an exhaustive generated client SDK.

Increment 0.3 additions

At increment 0.3 the API documented 38 paths, adding branch list/create/revise. `organization:view` and `organization:manage` are explicit grants. Branch configuration shares the idempotency and atomic audit coordinator and serializes with financial commands; no delete or balance-changing endpoint exists. Session-cookie configuration defaults strict; the hosted preview opts into Secure/HttpOnly/Partitioned cookies under a separate `__Host-erp_preview` name with CSRF unchanged. See preview/organization contract.

Increment 0.4 additions

At increment 0.4 the implemented specification had 43 paths, adding company profile/name revision and warehouse list/active-parent options/create/revise. `organization:profile:manage` is separate from branch management; warehouses use `warehouses:view` / `warehouses:manage`. No location migration, stock opening, balance, company lifecycle or legal-identity change is exposed. Warehouse deactivation is blocked by any current operational reference until a tested operational close workflow replaces the conservative guard. See the design and verification contract.

Increment 0.4.1 preview recovery

44 paths including public GET `/auth/config`. An explicitly enabled development-only transport negotiates a memory session with `X-ERP-Session-Transport`: memory at password login and sends `X-ERP-Session` on subsequent requests. The same session, CSRF, RBAC and scope checks apply. Invalid/disabled header credentials cannot fall back to a cookie. Authentication responses are no-store. Default remains cookies; production startup rejects `ENABLE_PREVIEW_MEMORY_SESSION=true`. See security tradeoff, lifecycle and executed evidence. The earlier local partitioned-cookie test did not resolve the user's actual Arena/browser issue.

Increment 0.5: controlled account changes

48 implemented paths. GET `/admin/accounts`, `/admin/accounts/parents`, `/admin/account-changes` require `accounts:configuration:view`; POST `/admin/account-changes` requires `accounts:propose`; POST `/admin/account-changes/:id/decide` requires `accounts:approve`, a different actor, CSRF and idempotency. Proposals are strict CREATE/RENAME unions and never accept creator/reviewer fields. No configuration-only grant implies journal posting. See the full state/permission/atomicity contract.

Implemented initial product endpoints (017)

Current OpenAPI contains 76 paths. Seven new admin routes: GET `products`, `products/:id` and `products/lookup`; GET/POST `product-changes`; POST `product-changes/:id/decide` and `products/:id/tax-preview`. `products:view` enables bounded reads and stored-price informational preview, not stock operations. `products:propose` and `products:approve` are separate, with current `products:cost:view` additionally required for approval. Cost-free readers receive null `purchase_price`; product audit snapshots require the cost grant even with `audit:view`. Authorized product audit price fields are projected as exact strings before JSON parsing; original native audits remain immutable. All supplied actor/balance/rate/preview-price fields are rejected. The moving weighted-average helper is internal, not an API.

Implemented stock endpoints (018)

GET `/inventory/{adjustments,balances,products,warehouses,reconciliation}` and source detail require `inventory:view` plus `inventory:cost:view`. Bounded paging/search and assigned branches apply. POST `adjustments` requires `inventory:create`; source `submit/refresh`, `approve/reject`, `post`, `cancel` and `reversal-drafts` require corresponding explicit permissions and cost visibility. Source actors resolve from the authenticated session; CSRF, stable idempotency, strict bodies, independent policy decisions and native guards remain authoritative. Reversal accepts date/reason only and negates original costs, not current average. Manual document routes exclude stock; stock audit snapshots are redacted without cost permission. OpenAPI now 89 paths; decimal payloads remain strings.

Implemented customer registration endpoints (019)

GET /customers, /customers/changes, /customers/:id, /customers/accounts and /customers/tax-categories require customers:view. POST /customers/changes and /customers/changes/:id/decide require customers:propose and customers:approve respectively, CSRF/idempotency/current company authority and independent review. No actor, balance, arbitrary currency or posting override fields are accepted. Generic audit masks customer snapshots without customers:view. Company-scoped master registration and AR-account metadata do not grant AR posting or generic financial reads. Current OpenAPI has 95 paths.

020 reviewed customer amendments

GET /customers/:id/amendments requires customers:view and supports bounded page/limit. POST /customers/:id/amendments requires customers:amend:propose; strict complete name/contact/limit/terms/hold fields plus expectedVersion/reason. Additional customers:credit:propose is required when any credit-policy value changes. POST /customers/amendments/:id/decide requires customers:amend:approve and, for credit-policy requests, customers:credit:approve. Maker cannot approve/reject their own request. No client actor, account, tax identity, balance, currency or active flag accepted. Used-customer credit changes fail closed. PENDING/APPLIED/REJECTED history is immutable; original registration stays frozen, current version advances exactly once. New audit entity CUSTOMER_AMENDMENT is redacted without customers:view. No endpoint here authorizes an invoice or creates AR.

Delivery gates, test strategy and operations

Phase order and acceptance

Phase	Build	Required exit evidence
0 Design	28 artifacts, target schema, explicit invariants	Executable DDL on PostgreSQL; design coverage and unresolved decisions
1 Core	identity, RBAC/scope, organization, attribution/audit, periods, COA, numbering, journal posting	DB-enforced balance/immutability; unauthorized/cross-company tests; repeat/concurrent posting; creator/approver/poster trace; security lifecycle tests
2 Inventory	catalog, searchable barcodes, UOM/prices/tax category, movement/valuation/reservation, transfers/counts	Concurrency no oversell, ending value at zero qty, movement=GL, approved adjustments, serial/batch controls
3 Purchasing	requisition → PO → approval → partial GRN → match → AP → payment	No double inventory, GRNI reconciliation, overreceipt/duplicate invoice/variance controls
4 POS/sales	cart → tender → stock → tax → journal → receipt	Cash/credit/walk-in/mixed tender atomicity, permissions/discount, receipt retry
5 Customers	accounts, allocation, credit limits, statements/aging	AR reconciliation, serialized credit checks, advance/FX policy
6 Returns	partial/full sales and supplier returns, notes/refunds	No excess cumulative return; original cost/tax; reversal dependencies
7 Expenses/cash	approvals, petty cash, drawer reconciliation	All floats/transfers funded, variance reviewed, drawer=GL movements
8 Tax	complete versioning, reports, Kenya/eTIMS mapping	Historical rules unchanged, categories/eligibility/tax point incl. advances; fiscal certification gate
9 Integrations	verified M-Pesa/card/bank adapters	Sandbox success/failure/timeout/late success/duplicate/reversal; settlement matches
10 Reports	operational/financial/user exports, reconciliation, close	Every control account reconciles, filters/exports agree, performance and UAT

Dependencies are real: minimum tax/pricing/AR/payment contracts must exist before sales/purchasing posting. Phase 8 expands tax compliance; it does not defer basic tax correctness until after sales. Phase 5 expands customer functionality; Phase 4 credit checkout remains disabled until AR/credit services pass their tests. No placeholders may silently post zero tax, zero COGS or unverifiable payments.

Before each domain implementation

Write workflow, entities/FKs, state graph, permissions/scope, attribution, policy/approval version, inventory/subledger/payment/tax/GL effects, reversal dependencies, audit/redaction, endpoint contracts and tests. Implement backend and database constraints first; run integration tests; only then implement UI and end-to-end tests. Update STATUS with actual evidence rather than declaring the whole module complete because tables exist.

Test pyramid

- Unit: Decimal arithmetic, tax incl/excl and discount apportionment, account rule specificity, workflow transitions, credit exposure, weighted average/last-unit residual, approval thresholds.
- PostgreSQL integration: migrations from empty DB; checks/FKs/RLS; deferred balance constraints; period lock race; journal immutability; rollback at injected failures; posting uniqueness; idempotency hash mismatch; negative inventory/over-allocation prevention; actor membership and attribution; config history.
- HTTP integration: missing/expired/revoked sessions; CSRF; rate limits; inactive user; invalid permission/company/branch; forged creator rejected; request bounds; state conflicts; source drilldown; no leaked credentials/SQL.
- Concurrency: synchronized parallel workers attempting last stock, same idempotency key, same document, same receipt/provider ID, same credit limit, same invoice allocation, period close vs posting. Assert final authoritative ledgers rather than only HTTP status.
- Property tests: generated tax/discount/cost values at precision boundaries; debit=credit per event; every return sum ≤ original; movement reversal net quantity/value; permutations of callbacks give one receipt.
- E2E: three distinct users create/approve/post, view trial balance and source audit; cashier opens/checkout/reprints/reconciles; warehouse partial receive/transfer/count; manager exception review. Playwright desktop/tablet and keyboard-only paths.

- Reconciliation: AR, AP, inventory, GRNI, tax, payments, drawer, advances, allocations, GL. Inject a deliberately missing effect into disposable test DB and verify exception detection. Empty ledger equality is not coverage.
- Security/load: dependency scan, SAST/DAST, tenant-isolation fuzzing, upload attacks, reset enumeration, session fixation, XSS/CSRF, permission escalation, payment spoof/replay. Load checkout at agreed volume with repeatable fixtures; record p95/p99 and lock time.

Required operational fixtures: cash/credit/walk-in/mixed sale; cash/bank/card/M-Pesa receipt; partial/full return and refund; partial/full GRN; supplier match/variance/return/payment; advance/unallocated allocations; transfer with discrepancy; blind count; expense with eligible/ineligible tax; multi-user attribution. Each successful fixture asserts all downstream effects and all failure fixtures assert none committed.

Production readiness gates

1. Accountant signs account/rule/tax mappings and opening balances; trial balance and every control reconcile.
2. KRA integration and tax classifications validated against current authoritative specifications; no compliance claim based solely on a VAT calculation.
3. M-Pesa merchant credentials and authenticated verification/settlement validated; secrets managed outside source.
4. RBAC/segregation approved, MFA/step-up, password reset delivery and session lifecycle independently tested.
5. Backup + PITR restore drill on separate instance, documented RPO/RTO, disaster-recovery ownership.
6. Migration rollback/forward plan; month/year close, fiscal failure and payment ambiguity runbooks exercised.
7. Load/concurrency/soak and independent penetration test; actionable observability and pager route.
8. PII retention, audit WORM export, attachment scanning and export/download audit approved.
9. Signed UAT with at least one complete procurement-to-cash and return/refund cycle per company.
10. Deployment keys/cookies/TLS restricted, DB runtime role non-owner, no seed/demo credentials and no exposed management ports.

Suggested objectives for owner agreement, not verified guarantees: RPO $\leq$ 15 minutes; RTO $\leq$ 4 hours; online checkout p95 < 1 second excluding provider round trips; primary financial commands auditable 100%. No benchmark claims are made for the delivered foundation.

Operations

Structured logs correlate request/source/journal/payment/provider event IDs without PAN, PIN, passwords or tokens. Metrics: posting failures, deadlock/retry counts, outbox age, unreconciled amounts, fiscal failures, pending payment age, audit export lag, auth anomalies. Alert on imbalance attempts even when rejected. Retry serialization/deadlock failures at transaction boundary with bounded backoff; never retry external initiation blindly.

Outbox workers claim jobs using SKIP LOCKED leases, bounded retries and dead-letter review. Object uploads and provider calls are outside DB transaction; orphan quarantine objects expire through audited lifecycle jobs. Reports use repeatable-read cut-offs or read replica watermark; no financially inconsistent mixed snapshots. Close export jobs retain filter/schema version, actor and content hash. Excel/CSV escape formula injection; PDF assets are local and print layouts repeat headings.

Decisions to confirm before corresponding release

Legal company names/PINs and VAT registration; branches/warehouses/terminals; actual product tax mapping; opening stock cost and stock cut-off; approval thresholds and required separation; returns windows/quarantine rules; bank/M-Pesa/card settlement accounts and merchant contracts; currencies and FX source; fiscal provider mode; hosting/RPO/RTO; retention and email/MFA providers. These do not block blueprint/core tests but block live operation.

Increment 0.2 evidence

Access administration, recovery/session lifecycle, threshold/quorum approval snapshots and source revision controls were implemented under the separate preimplementation contract. Backend tests precede browser acceptance; the two current Chromium evidence files cover accounting and controls/recovery respectively. Production SMTP, MFA, organization/COA/rule/numbering controls and operational modules remain open. Exact current counts and limitations are maintained in STATUS, not inferred from the target schema.

Increment 0.2 — Phase 1 access and approval controls

Implementation contract, written before code. Scope: strengthen the existing accounting kernel, not add operational CRUD or enable live inventory/payment posting.

Workflows, permissions and financial effects

Workflow	Command / permission	Effects	Reversal / history
Provision a new human	users:manage	New global identity with random unusable initial password, company membership, explicitly delegated roles/branch scope, audit. No balance or GL effect. Password established through reset. Existing identities cannot be silently attached/taken over.	Membership deactivation, not deletion
Change company access	users:manage, expected membership version, reason	Active membership, roles and allowed branches replaced atomically. Cannot change self, grant outside explicit delegation or remove the last active users:manage administrator.	New audited access change; identities and historical documents remain
Create/revise role	roles:manage, expected role version	Permission set must be within explicit per-administrator delegation allowlist, including existing grants on revision. Cannot edit a role the acting administrator holds. All users retain historical activity.	Versioned audit of old/new grants; no role deletion
Revoke own session(s)	Authenticated session + CSRF	Single-session or all-session revocation, security event. Does not change another company's membership.	Re-authenticate; revoked tokens never revive
Request/reset password	Public generic response; limited by email digest and trusted IP	Durable encrypted delivery record, hashed expiring single-use token. Consumption atomically changes password, consumes outstanding reset tokens and revokes every session.	Reset again; original requests/completion retained in security audit
Draft policy	approvals:manage	New immutable policy version, functional-currency threshold and ordered permission steps. No ledger effect.	Supersede with independently published new version
Publish policy	approvals:publish, different human than creator	Activate policy; supersede previous active version at the same threshold. Does not affect already submitted documents.	Publish another version; never mutate historical rules
Submit source	journals:create, original creator	Resolve highest active threshold not exceeding debit total. Snapshot policy, amount, separation flags, required steps and revision. If no configured threshold applies, use explicit company-baseline one-step policy snapshot.	Reject/revise/cancel before posting
Approve source	journals:approve plus active step's permission	Append decision; move APPROVED only when all ordered quorums satisfied. Distinct humans across a revision. Poster exclusion considers every approver, not only the final approver.	Reject before full approval, or revise; prior decisions retained
Revise source	journals:edit, original creator, expected revision, reason	Increment revision, archive old contents/approval metadata, clear effective approvals and return DRAFT. No journal effect. Reversal quantities/accounts remain immutable.	Another revision; posted/cancelled sources cannot be revised
Reject/cancel source	journals:reject / journals:cancel, expected revision, reason	Controlled transition with actor/time; no financial effect. Posted sources cannot be cancelled.	Rejected → new draft revision; cancellation terminal

Role names are not authorization rules. Delegation grants are provisioned explicitly, not inferred from an administrator's own posting powers. Administrators cannot assign themselves roles, edit their own roles or grant delegation rights through these APIs. Changing a company membership does not globally deactivate an identity or revoke sessions used for other companies. Global user deactivation/identity federation is outside this increment.

Transaction and concurrency model

Extract a shared CommandService from AccountingService so administrative commands use the same idempotency and audit boundary without depending on accounting controllers. Financial/workflow commands take a shared company authorization lock; access/role changes take its exclusive form. Revalidate live session, active identity, membership, required permission and branch scope inside that boundary. Hold share locks on actor identity/session through commit so completed revocation/password reset cannot leave a later command posting with a stale session. Consistent lock ordering and integration tests are mandatory. No provider calls inside these database transactions.

Password reset and login lock the same user before updating credentials/sessions. Reset token consumption locks user then token to prevent deadlocks and duplicate consumption. Public requests always acknowledge generically; bounded rate-limit buckets are enforced atomically. Rate-limit data is not an accounting balance. Public requests are attributed to the AUTH_SERVICE principal, not to the unverified email's human owner.

Schema changes (additive migration 004)

- Global append-only `security_events`, service principal, `auth_rate_limits`, encrypted `auth_delivery_outbox`.
- User/member/role version and attribution fields, explicit company-scoped `permission_delegations`.
- Existing approval policy/step tables promoted to immutable published configuration; decisions extended with step number and revision-level distinct-actor uniqueness.
- `source_documents.approval_snapshot`; append-only `source_revisions` archive old contents, approval metadata and editor/reason.
- Update source state/approval guards. Preserve all existing posted documents and journals; do not replace applied 001–003 migrations.
- Runtime grants restricted to supported core/control tables. New domain services still cannot write inventory, payments, tax or subledgers.

Reset delivery boundary

No reset token is returned in public API responses, written in audit, logged or stored plaintext in database. Use AES-256-GCM with a separately configured 32-byte key; only its hash belongs in the token table. The outbox worker can deliver via configured SMTP or an explicit development file sink with private filesystem permissions. Delivery is at-least-once (a crash after sending may resend the same link, never mint a second token); consumption is single-use. Do not claim production email/MFA is configured. If no delivery key/adaptor is configured, disable the feature clearly rather than silently pretending reset email was sent. Email templates use a configured trusted web origin; never the incoming Host header. Expired jobs must not send usable links.

UI and API

Add a Control center for members, roles, delegation visibility, policy versions and security events; add own-session management and password reset/change screens. Company financial pages remain permissioned. Administrative users do not receive journal-posting rights by default. Show policy/revision decisions on source timeline; revision/cancellation/rejection dialogs require reasons and expected version. Search/paginate member lists; role/permission/account selection is searchable and bounded.

New API families: `/admin/users`, `/admin/roles`, `/admin/permissions`, `/admin/approval-policies`, `/admin/security-events`; `/auth/sessions`, `/auth/password-reset/request`, `/auth/password-reset/consume`, `/auth/password/change`; `/documents/:id/revise|reject|cancel`. All write DTOs reject supplied actor fields. Financial journals remain immutable.

Required tests before UI

Existing accounting tests remain green. Add: read-only and company-isolation checks; no self-escalation; no permission grant beyond delegation; stale access/role version; last-admin protection; deactivated membership denial; revoked/expired session denial; single-use/expired/concurrent reset; generic unknown-email response; no plaintext reset secrets in DB/audit; reset invalidates sessions; change-password current-password check; reset rate limits; independently published policy; threshold selection; ordered quorum and unique approvers; creator/poster exclusions across all approvers; policy change cannot alter submitted snapshot; revision preserves creator and all prior decisions while invalidating effective approvals; posted/cancelled edits blocked; durable failure audit; idempotent admin retry; database constraints prevent bypass. Browser-test the actual control-center workflow after backend tests.

Remaining release gates: MFA/step-up, production SMTP/secrets/WORM/monitoring, organization/COA/rule/numbering administration, complete operational close controls, penetration/load/restore tests. Phase 1 remains open until these are addressed; Phase 2 inventory must not bypass the accounting boundary.

Implementation and verification checkpoint — 26 September 2026

Backend and frontend for this contract are implemented in increment 0.2.0. A shared command coordinator, additive migration 004, recovery worker and two Chromium acceptance scripts accompany the release. Real PostgreSQL/HTTP tests pass, including a separate delegated manager's unsuccessful attempt to remove the last user administrator. Accounting browser coverage now includes revision archival. Control browser coverage includes independent policy

publication and actual encrypted-outbox → private-file → reset → login → revoke-all delivery, not just database token inspection. See test-results.txt, browser-test-result.json, browser-controls-result.json and ../STATUS.md for exact evidence and remaining gates.

Administrative catalogs are bounded rather than fully searchable in this increment: members support server-side search/pagination, roles/policies/branch options and sessions cap at 100. UI forms do not grant the administrator delegation rights they do not already hold. Provisioning an existing global identity, global account deactivation and delegation-owner administration remain deliberately outside the company administrator's API.

Recovery links are stripped from the address bar on initial load and on an in-tab hash navigation. Login requires application/json. Worker success/failure logs omit mail content and recipient address. Production SMTP, key rotation/retention, dedicated worker privileges, MFA and release security review are not completed by the development-file delivery test. Modal focus trapping/accessibility and broader network-failure browser coverage remain open.

Increment 0.3 contract — preview authentication and branch configuration

Written before implementation. Phase 1 stays open. No operational balance or stock writes.

Preview authentication correction

Observed: a successful login is in server history during the user's reported failure. Current session cookies are SameSite=Strict, which are unsuitable for a cross-site embedded preview. A raw e2b hostname also returns a platform traffic-token 403 without Arena's preview authorization; do not publish a raw hostname as a universally usable login link.

Reproduce cross-site iframe authentication in Chromium using a local HTTPS reverse proxy and a separately hosted parent origin. Add an explicit SESSION_COOKIE_MODE=partitioned deployment option: HttpOnly, Secure, SameSite=None, Partitioned. Default stays same-origin strict. Reject insecure partitioned configuration. Never expose the session token in JSON or browser storage, weaken CSRF checks, or add permissive CORS. Logout must use matching cookie attributes. A successful password followed by absent session must produce an actionable cookie/preview diagnostic, not misleading invalid-password advice. Use Arena's live preview/open-in-new-tab controls; platform traffic authorization is not an ERP credential.

Tests: strict default, secure partitioned opt-in, invalid insecure configuration, cookie flags, session survival/reload and logout in a genuinely cross-site iframe, CSRF still denied. Test the existing three-user accounting flow as a regression.

Branch configuration slice

Workflow: permissioned organization viewer searches current-company branches; an explicitly delegated organization manager creates or revises code/name/active state with a reason and optimistic version. Branch creation gives nobody automatic access. Existing branch-scope assignment remains the member administrator's workflow. No hard-coded role-name authorization.

Entities: existing company → branch → warehouse/source/journal relations. Add branch version/created_at/updated_by/updated_at; preserve original creator. Code is immutable once created. No delete command. Company/base currency, warehouses, COA, accounting rules and numbering configuration remain subsequent slices.

Permissions: organization:view and organization:manage; current-company membership required. Add permissions to catalog, not automatically to arbitrary existing roles. Demo seed explicitly grants the development administrator these permissions and delegation entries.

Commands: GET /admin/branches?q=&page=&limit=; POST /admin/branches; POST /admin/branches/:id/revise. Strict DTOs reject frontend actors and company IDs. Commands use the existing idempotency/atomic-audit coordinator and exclusive company configuration lock; accounting takes the shared lock, preventing deactivation/post races. New branch defaults active. New access is never inferred from administrative authority.

Effects: no GL, stock, tax or subledger effect. Creator cannot be overwritten. Revision records before/after and reason in immutable audit. Deactivation blocks new financial work and branch scope selection; reject deactivation if pending unposted sources or active warehouses exist. Posted history remains readable. Correction is another attributed version; no destructive rollback. Direct runtime SQL guards protect immutable code/company/creator and version increments.

Tests before UI: read-only denial, company isolation, impersonation/unknown fields, idempotent retry and changed-body conflict, duplicate code, stale version, creator preservation, pending-work/warehouse deactivation guard, inactive-branch posting denial, existing journal counts unchanged, audit coverage and SQL identity mutation denial. Then a browser create → revise → deactivate → reactivate flow; no dummy balances.

Verified implementation checkpoint

At the increment 0.3 checkpoint, these controls were implemented and deployed to the development preview. 49 backend tests passed; four browser workflows passed, including branch create/revise/deactivate/reactivate and a genuinely cross-site TLS iframe. The cookie names are distinct: strict erp_session, partitioned __Host-erp_preview. Tests also verify that an old cookie from the other mode cannot shadow the authenticated identity. Insecure production or partitioned configuration fails at startup. Matching logout cookie attributes clear the correct partition.

The two local HTTPS origins are 127.0.0.1 and localhost on an ephemeral test proxy. The first attempt using a public-looking synthetic parent was correctly stopped by Chromium's local-network access rules; the final test keeps both endpoints local but cross-site, without disabling browser cookie protections. Test private keys stay in excluded .cache. Arena's traffic token is never read or bypassed by this test.

Branch revision uses runtime-only updatable column grants and a deferred atomic-audit constraint; owner bootstrap is explicitly separate. Existing inactive scopes can be retained, never newly granted, and new members start with no branch scopes selected. Seed permissions are development fixtures, not a production delegation-owner API. Company/warehouse/COA/rule/numbering administration, MFA, production mail and full release security remain unfinished.

Integrated accounting acceptance extension

Branch-scoped source/journal numbers can repeat across branches. Read projections and the UI must display the immutable branch code alongside the number. Verify a journal in the newly configured branch through submission, approval and posting, then deactivate the now-idle branch and prove the immutable posting is still readable to already-scoped users but not an out-of-scope auditor. No journal is changed by deactivation.

Increment 0.4 contract — company profile and warehouse configuration

Design before implementation. Phase 1 remains open; these are configuration controls, not the inventory engine. No new company onboarding, stock movements, balances, tax or GL posting is enabled by this slice.

Company profile

GET /admin/company requires organization:view, uses the authenticated selected company, and shows code, display name, functional currency, timezone, tax PIN, status, version and attribution. POST /admin/company/revise requires the separate organization:profile:manage permission, expectedVersion and a reason; only the display name is editable. New-company provisioning/global owner authorization, legal/tax identity changes, lifecycle changes and functional currency/timezone changes are not exposed. Existing code, tax PIN, accounting/approval flags and origin are immutable under this interface. Do not silently reinterpret old journal currencies/dates.

Name revision takes the exclusive company authorization/configuration lock, retains original creator, stamps editor/time/version and writes before/after audit atomically. It has no financial effects. Correction is another attributed revision. Runtime grants restrict updates to permitted columns; a deferred audit constraint rejects unaudited runtime SQL.

Warehouse/location configuration

Company → branch → warehouse relations stay authoritative. Reuse the existing warehouses table. Separate warehouses:view / warehouses:manage permissions; no role-name checks. Company-scoped paginated search may filter by branch. Searchable active-branch options are permissioned through warehouses:view, not inferred from financial branch access.

Workflow: create code/name/location type under an active same-company branch → reasoned optimistic name/status revisions → deactivate/reactivate. Codes, company, parent branch, location type and original creator are immutable. No deletion, relocation, stock quantity, value, cost or opening-balance input exists. Moving stock must eventually use a movement document, not change a warehouse parent.

Types: SELLABLE, TRANSIT, QUARANTINE, DAMAGED, RETURNS. These are configuration categories; creating one does not implement sale eligibility or inventory valuation. Creation grants no user scopes and creates no journals or stock projections.

Commands: GET /admin/warehouses; GET /admin/warehouses/branch-options; POST /admin/warehouses; POST /admin/warehouses/:id/revise. All mutation DTOs reject extra actor/company/balance fields. Required reason, idempotency, version conflict, atomic audit and live permission checks reuse CommandService with exclusive company locking. Creation/reactivation requires an active parent branch. An active warehouse blocks branch deactivation through the existing branch guard, including concurrent changes.

Until operational lifecycle engines exist, deactivation is deliberately conservative: **any record referring to the warehouse in the current operational baseline blocks it**, even a historical or zero-balance record. Do not pretend to have implemented a safe operational close. The guard covers every current foreign-key reference: terminals, movements, balances, reservations, valuation policies, counts, transfers (all three locations), sales/return lines, requisitions/POs/GRNs/purchase return lines and accounting rules. A tightly scoped SECURITY DEFINER boolean function checks these without granting the runtime access to unreleased domain tables; fixed search_path, tenant/actor/permission checks, no dynamic SQL, no public execution. Future inventory/purchasing must replace this conservative gate with tested open-work/balance checks and use the same transaction/locking boundary.

Schema and audit

Additive migration 006: version and edit attribution on companies; version/creation/edit attribution on warehouses. Unknown creation dates of legacy warehouses remain NULL. Preserve all already-applied migrations and financial records. Runtime writes to operational domain tables remain denied. Database guards preserve identity/version/actor, active parent and atomic audit. Owner/DBA bootstrap remains an explicitly trusted maintenance boundary. Company/warehouse display-name changes do not rewrite historical origin IDs.

Tests before UI

Company: read/rename permission isolation; company isolation; reject currency/flags/actor spoof; stale version; idempotent retry/changed-body conflict; original creator preserved; unchanged GL and source counts; SQL column and audit guards.

Warehouse: permission/tenant/strict DTO checks; active-parent validation; duplicate code; idempotency; version races; no automatic scopes/projections; immutable code/type/branch; block branch deactivation while active; deactivate empty warehouse then branch; deny warehouse reactivation under inactive branch; reopen branch then warehouse; full FK coverage assertion on conservative dependency guard; restricted helper invocation; representative terminal/valuation dependencies (owner-only test fixtures); runtime cannot write inventory balances; audit rollback. Race-test warehouse creation vs branch deactivation and assert no active child under inactive parent, whichever command wins.

After backend passes, browser-test company rename/restore, warehouse create/search/rename/deactivate/reactivate, inactive-parent/dependency feedback, read-only reviewer and existing branch/financial/embedded-login regressions. Document exact evidence and remaining phase gates.

Verification checkpoint — 26 September 2026

The primary company/warehouse tests were written before implementation, and the initial backend suite passed before the UI was added. Additional security/concurrency cases bring the current suite to: **57 passing tests, zero failures**. The restricted helper is checked for non-public execution and fixed search_path; every warehouse foreign-key reference in the current schema must appear in its static dependency checks. Owner-only valuation-policy and inactive-terminal fixtures prove conservative deactivation denial; application runtime inventory reads and writes remain denied. Concurrent warehouse edits and warehouse creation versus branch deactivation are tested on real PostgreSQL.

The new Chromium organization workflow passed together with the existing accounting, branch, controls/recovery and secure cross-site iframe workflows. The organization test restores the demo company's original name through another attributed revision, rather than erasing audit history. Screenshots and exact outcomes are in browser-organization-result.json and screenshots/warehouse-configuration.png. The sandbox cookie fix remains in place; use Arena's live preview entry, not a raw traffic-protected hostname. Production certification is not implied.

Preview session recovery — 0.4.1

The user still receives accepted-password/missing-session errors after partitioned cookies were enabled. Local cross-site cookie tests did not prove cookie delivery through the user's actual preview/browser/proxy. The precise external failure point is not confirmed; do not claim it is fixed by asking the user to reset a valid password.

Bounded correction

An explicit `ENABLE_PREVIEW_MEMORY_SESSION=true` flag enables an alternative session transport only in development; enabling it with `NODE_ENV=production` must fail startup. Default remains cookie-only. The client opts in with `X-ERP-Session-Transport: memory` at password login; only successful password authentication may return an opaque `previewSessionToken`. This is the same hashed, revocable, expiring server session as cookie authentication—not a demo-user bypass or new authority.

In this mode the client sends `X-ERP-Session` on same-origin API requests. The header is authoritative: invalid or disabled header authentication never falls through to a valid cookie. All existing CSRF, RBAC, membership, segregation, idle/absolute expiry and command revalidation still run. Do not allow permissive CORS. Authentication responses must be no-store. No token may be put in a URL, `localStorage`, `sessionStorage`, audit, telemetry or logs. Login tokens are JavaScript-readable while in memory, so this is not equivalent to `HttpOnly`-cookie protection and must not be presented as production authentication.

`GET /auth/config` reports only whether this development option is enabled. When enabled, the UI does not automatically restore cookies after a reload, preventing stale cookie identity from replacing a lost in-memory session. Successful preview login expires the previous configured cookie and does not issue a new cookie. The UI shows an explicit reload/sign-in notice; logout and revocation clear the memory credential. A late response from an earlier authentication attempt must not replace or clear the current session.

Verification

Write backend tests for default/production fail-closed configuration, successful password-only token issuance, invalid-password denial, no token in normal login, cookie-free authenticated reads, authoritative header identity, tenant/RBAC and CSRF denial, expiry/logout revocation and no-store responses. Browser-test the actual UI behind a test proxy that strips every `Set-Cookie` and `Cookie` header, not a test that manually supplies a cookie. Verify control-center navigation, a CSRF-protected session revocation, reload requiring reauthentication, and no session credential in browser storage. Re-run the cookie-only test with the memory negotiation removed to preserve coverage of both modes. Real Arena/browser confirmation still requires the user's retry.

Executed evidence — 26 September 2026

- Red tests failed before implementation (missing no-store/transport policy). Backend implementation then passed; final expanded suite: **60 tests, 60 passed** on Node 22 and PostgreSQL 17.
- `docs/browser-preview-memory-result.json`: ten Chromium UI/security checks passed in a real local cross-site TLS iframe with all `Cookie/Set-Cookie` headers stripped; zero page errors.
- `docs/browser-embedded-result.json`: separate ephemeral API with the flag disabled; real `HttpOnly/partitioned-cookie` login, reload, CSRF denial and logout passed. Does not alter the running sandbox configuration.
- `docs/browser-organization-result.json`: all twelve configuration workflow checks passed using the memory transport.
- Finance regression initially timed out twice at revision dialog dismissal without a failed server response. Diagnostic and unchanged-script reruns passed, latest source JDR-2026-000005. Root cause of the intermittent UI timing remains open; do not misrepresent the suite as consistently green or this patch as fixing that issue.
- The running development preview uses the new mode. **Actual Arena traffic authorization and the user's browser are not accessible to these tests; user retry remains the final environment-specific confirmation.** No valid password reset is needed.

Loading-state regression caught during final retest

A post-revocation login stalled when an old workspace refresh was still pending. A deterministic test now holds that refresh across self-revocation; it failed before the correction (sign-in remained disabled as "Signing in..."). Background refresh and mutating-command loading states are now separate, and clearing the local session resets both. The test passes without retrying the click or sleeping to hide the race. The finance workflow passed afterward; its earlier revision-dialog timeouts are retained above rather than retroactively claiming their exact cause was proven.

0.5 — controlled chart-of-accounts additions and naming

Design before implementation

This increment adds the first COA configuration commands to the existing accounting engine, not a disconnected account CRUD screen. Scope: propose a non-control account/header, propose a display-name correction, independently approve/apply or reject. No account deletion, reclassification, code change, currency change, reparenting, active-status change, opening balance, control-account creation or posting-rule changes. Those remain separate release gates. Existing posted journal amounts/account IDs/source IDs are not rewritten. Reports resolve the current account display name; the configuration audit preserves earlier names (historical-name-as-of reporting is not claimed).

Workflow and state machine

1. Authenticated proposer submits CREATE or RENAME with a reason. A pending request is immutable; it is not an active account and has no financial effects.
 2. A different authenticated reviewer with accounts:approve reviews the complete request. APPROVE atomically applies it to accounts, marks APPLIED, and records request/account before and after. REJECT records reason and reviewer without changing any account.
 3. PENDING -> APPLIED or REJECTED only. No editing, deleting, resubmitting or resetting terminal requests. Corrections require a new linked-to-account request. No retrospective journal correction is implied. A stale rename must be rejected and replaced; it cannot overwrite a newer version.
 4. Same idempotency key/payload returns the original result. Changed payload or repeated terminal transition with a different key conflicts. Competing reviews serialize; only one transition/audit/account mutation commits.
-

Data and relationships

account_change_requests: company, kind, immutable normalized proposal columns, target account + expected version for rename, preallocated account UUID for creation, immutable created_by/created_at/reason, state, reviewer/time/decision reason and applied account reference. All account references use composite company foreign keys. accounts gain version, original creation timestamp (NULL for legacy rows), last editor/time and last applied request FK. Creation preserves original human proposer in accounts.created_by; applying reviewer is retained on the request. RENAME preserves original account creator and protected metadata.

New accounts inherit the company functional currency policy (currency NULL means unrestricted by an account-specific denomination, but this release's journal engine still enforces company functional currency). Parent is optional; if present it must be active, same-company, non-postable, non-control, same classification and compatible currency. Parent/type/postability are immutable through this API; ancestors cannot be changed into posting accounts. Both postable non-control accounts and non-postable headings can be proposed. Code is uppercase 2–24 characters, unique per company when applied; pending requests do not reserve codes. Name 2–120 characters; reason 5–500. No float/money input exists here.

Permissions, approval and concurrency

- accounts:configuration:view: paginated account catalog, paginated request queue and parent search.
- accounts:propose: submit proposals; does not grant approval or journal posting.
- accounts:approve: approve/reject another person's request; does not grant proposal or financial posting.
- Existing accounting:view continues to authorize the financial account catalog.

Backend checks all grants; no role-name checks. Demonstration grants: Esther proposes, Felix reviews; finance/audit users may view. Users with both permissions still cannot review their own requests. Configuration commands take the existing exclusive company advisory lock; financial commands take its shared counterpart. Permissions/membership/session are rechecked after locking. Approval revalidates duplicate code, parent eligibility or target version. Stale/competing requests fail without side effects. Database triggers enforce request immutability, review separation, matching applied proposal and deferred same-transaction audit. Runtime account writes without an approved matching request fail. Trusted owner/bootstrap is explicitly outside runtime authorization, not a public bypass.

Effects and reversal

Propose/reject: only configuration request, idempotency result and audit. Approve: these plus one new account or one versioned name change. No journal, source document, sequence, balance or inventory movement. New postable accounts immediately participate in the existing journal draft -> independent approval -> independent posting workflow, with exact decimal validation and normal double-entry/reconciliation checks. Reversal of a financial transaction still uses linked opposite-entry journal drafts; a naming correction uses another independently reviewed request.

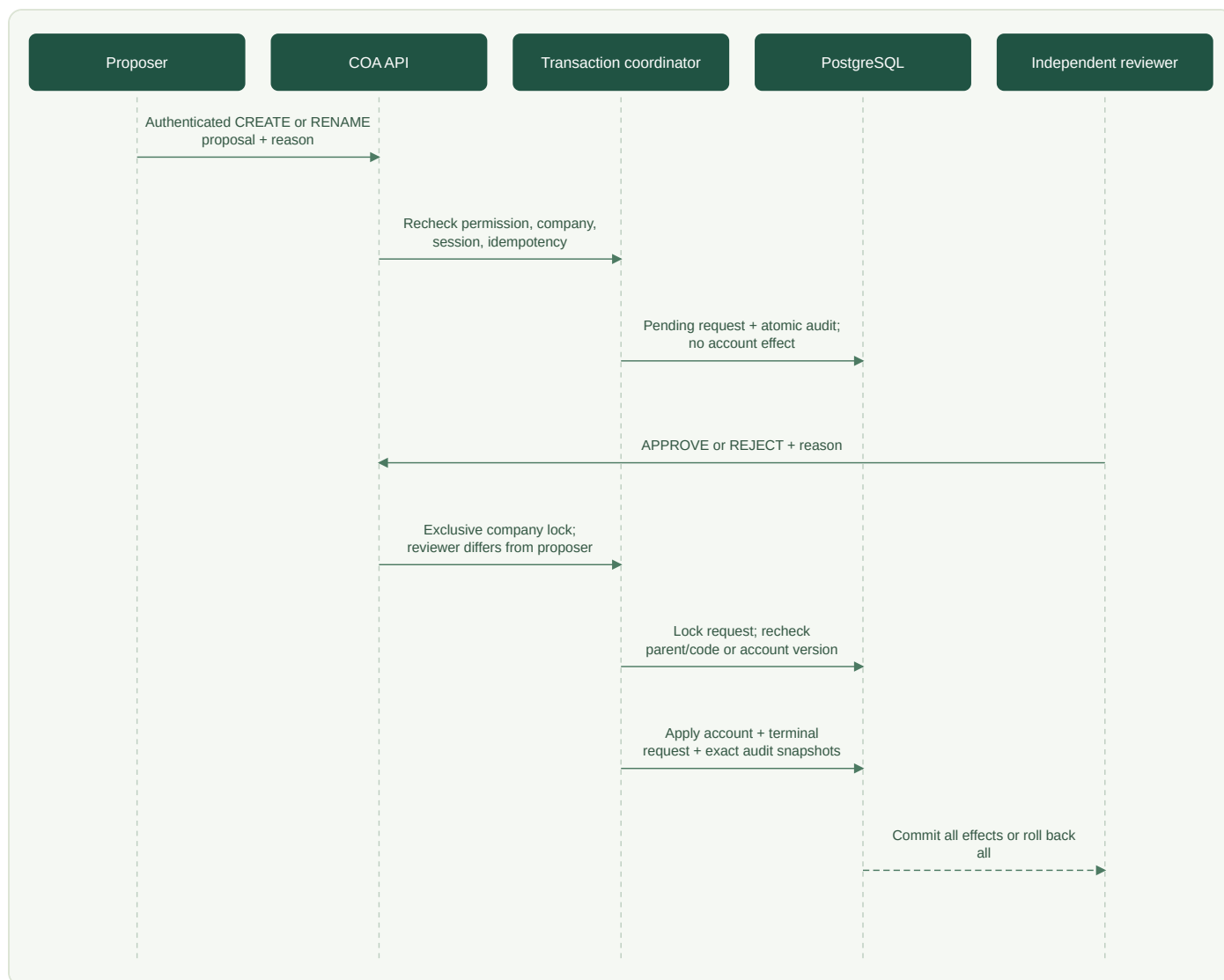
API

GET /admin/accounts?q=&page=&limit= : paginated complete catalog and original/editor attribution. GET /admin/accounts/parents?accountType=&q=&page=&limit= : eligible parent search. GET /admin/account-changes?q=&state=&page=&limit= : paginated full review payload/attribution. POST /admin/account-changes : strict discriminated CREATE/RENAME body, Idempotency-Key, company and CSRF. POST /admin/account-changes/:id/decide : {decision: APPROVE|REJECT, reason}, same headers.

Acceptance tests before frontend

Authentication/CSRF/RBAC/company isolation; proposer cannot self-review even with both grants; pending request creates no account or journal; parent eligibility/cross-company rejection; unknown/forged actor/protected fields rejected; atomic approve + audit with proposer/reviewer attribution; replay versus conflicting idempotency; rejection no account effect; concurrent reviews one winner; conflicting code proposals; competing rename versions; immutable/unaudited direct runtime writes denied; transaction rollback on audit failure; newly approved account flows into a balanced independently posted journal and trial balance. Browser proof covers maker/reviewer navigation, search, approval/rejection, journal account picker availability, and sandbox sign-in regression. No production certification is implied.

Review/apply sequence



Executed verification — 26 September 2026

The preimplementation HTTP tests first failed against the absent routes. After implementation, **76 total backend tests passed**, including 15 COA cases plus their suite. COA coverage includes audit-failure rollback, competing approval/code/version conflicts, direct runtime write denial, tenant isolation, current-grant removal, header-account posting denial, maker/reviewer separation and exact journal integration. Existing financial/auth/control tests also pass.

docs/browser-coa-result.json records 16 passing Chromium checks and zero page errors: proposal/parent search, no pending-account leakage, separate proposer/reviewer access, independent creation and label revision, rejection without account effect, actual journal autocomplete -> source -> approval -> posting -> trial balance, auditor read-only access, Escape handling, dark mode and 390px layout. The first browser run caught an ambiguous test locator when two applied requests shared an account code; the locator now also matches change kind. The application keeps both immutable requests, rather than deleting history to satisfy the test.

Cookie-free (ten checks) and cookie-only (five checks) browser regressions passed, along with all prior organization, branches, access/recovery and journal browser scenarios. External SMTP and the user's actual Arena traffic authorization remain outside these tests. SQL schema export: 116 tables, 1,212 columns and 490 FKs; the new public OpenAPI has 48 paths. This remains a Phase 1 development increment, not a production-ready operational ERP.

Next.js host adaptation

The new execution specification explicitly requires Next.js. Reuse the current React workspace and its tested business screens; replace only Vite's hosting/bootstrap with Next.js App Router. No accounting/authentication rewrite. The browser still fetches relative /api/v1 routes; server-side rewrites proxy to the private Nest service. Runtime session state stays client-side; neither password nor memory session is serialized into server HTML. A client-only dynamic boundary is necessary because the existing workspace uses browser storage/crypto during initialization.

Development listens on 0.0.0.0:5173, preserving Arena preview links and regression scripts. Allowed development origins are the preview host family and the local cross-site test origins, not permissive API CORS. Production uses a non-root standalone Node image and a nonce-based CSP with frame-ancestors none. Production authentication remains HttpOnly-cookie-only; the Nest startup guard still forbids preview memory transport.

Verification gates: TypeScript/build; production server HTML and API proxy smoke test; all existing browser suites against the Next host; both cookie-independent development and cookie-only authentication regressions. A successful Next build is not evidence that Docker or the unreleased operational ERP is complete.

Central document-number allocator

This dependency step extracts number-format defaults from financial service code into a database registry and makes the existing sequence counter monotonic. Company/branch/year proposals and their independent approval UI are now implemented and tested; see the configuration contract.

Registry: released document type → initial prefix and padding, versioned by reviewed migrations. Current initial defaults preserve JDR/REV/JE and six digits. Existing sequences retain their prefix and padding forever. New sequence rows use the shared resolver: an approved branch/year override, otherwise the registry default; unsupported types fail closed. No client chooses a document number or trusted actor. Approved branch/year overrides enter that same resolver and guard, not a second numbering engine.

Counter key remains company + branch + document type + calendar year from business date (the historical SQL column is named `financial_year`). Number format is prefix-year-zero-padded counter. This is not arbitrary fiscal-year numbering. Allocation is inside the source/posting transaction, uses INSERT ON CONFLICT UPDATE locking, and rolls back with any downstream failure. Bigint counters remain strings when formatting. Capacity exhaustion fails visibly; it never wraps or truncates.

Database guards: runtime cannot change sequence identity, format or padding; every UPDATE increments by exactly one. The deferred commit guard requires the newly allocated number to correspond to a source/journal created in that same transaction by the transaction actor. This catches abandoned/forged counter updates. Existing authoritative source/journal immutability and uniqueness remain in force. The database owner remains the trusted migration/recovery principal, not an application login.

Tests precede implementation: registry-driven new-year formatting; stable existing-year formats when defaults change; parallel unique allocation; downstream rollback leaves no used counter; direct decrement/prefix rewrite and abandoned allocation are denied. Configuration evidence additionally covers independent review, exact prior/after audit snapshots, publication racing first allocation, real financial posting/rollback, browser workflow and least-privilege navigation. There is no counter-setting API or approval bypass. A source number is fixed at initial allocation; controlled unposted date revisions do not renumber it.

Controlled document-numbering configuration

Implemented for the released local document types; not an operational ERP or fiscal-adapter completion claim. This contract was written before implementation and is now updated with actual behavior/evidence. The central allocator and CommandService are reused; there is no second counter engine.

Workflow, entities and states

Control center → **Numbering**. Choose an active company branch, released document type and calendar year; propose a prefix, minimum counter padding, current expected version and reason. Pending proposals make no effective change. A different authorized human applies or rejects with a reason. PENDING → APPLIED/REJECTED is terminal. A subsequent correction requires another immutable proposal and a new version.

document_numbering_policies stores the current approved override and original creator, current version, updater and last request. numbering_change_requests preserves every proposal/review and the generated target ID. Composite tenant FKs, RLS, approved-request matching, optimistic versions and deferred exact before/after audit checks reinforce backend controls. No journal, stock, tax or account balance is changed by configuration.

Registry defaults are the fallback; branch/type/year overrides take precedence. New sequences copy that resolved format; established sequences retain their original format. A key becomes permanently format-locked when its sequence is initialized, including cancellation of the resulting draft. No user endpoint sets, decreases, restarts, wraps, deletes or reserves a counter. An initialized legacy zero counter is also conservatively locked. Minimum padding does not truncate larger bigint values.

The source number is fixed at initial allocation. Controlled edits of an unposted source do not renumber it; its revision history retains earlier dates/content. Calendar-year numbering is not an arbitrary fiscal-year format grammar. External fiscal/eTIMS identifiers and operational document registrations remain separate unreleased scopes.

Authority and concurrency

- numbering:view: effective format, catalog, type registry, branch options and request history.
- numbering:propose: create a proposal, never apply it.
- numbering:approve: independently apply/reject; having both grants still cannot permit self-review.

These are explicit company-configuration grants, not role-name checks or implicit financial posting rights. The browser never supplies a creator/reviewer ID. Writes require CSRF, idempotency and current session/membership/grants. Company/branch/date/type/version are rechecked at approval. Configuration uses the existing exclusive company lock; allocation uses its shared counterpart. Prefix conflicts are checked against other effective types, established sequences and fallbacks within the same branch/year; uniqueness constraints back the checks.

A proposal does not reserve a prefix. Competing publications can make it stale or conflicting. A publication racing first issuance either takes effect before issuance, or fails because the sequence has initialized; it cannot rename the issued number. Direct runtime SQL cannot bypass matching approval, immutable fields or exact atomic audit. Unknown historical allocation actor/time remain NULL; new allocation metadata always records the actual transaction actor/time.

API

- GET /api/v1/admin/numbering-policies: paginated overrides and used/locked status.
- GET /api/v1/admin/numbering-policies/effective: branchId, documentType and year; resolved prefix/padding/version, inherited flag, lock and branch.
- GET /api/v1/admin/numbering-policies/types: released registry types.
- GET /api/v1/admin/numbering-policies/branches: server-searched/paginated active branches.
- GET/POST /api/v1/admin/numbering-policy-changes: paginated history or strict proposal.
- POST /api/v1/admin/numbering-policy-changes/:id/decide: APPROVE/REJECT plus reason.

The UI has paginated search, current-format loading/error feedback, typed fields, maker/reviewer attribution in Africa/Nairobi, independent review confirmation, stale-version display, immutable decision history, keyboard branch selection, focus containment/Escape and read-only/dark/mobile states. It reuses the existing uncertain-request idempotency retry mechanism.

Executed verification

`apps/api/test/numbering-configuration.test.ts` exercises authentication, grants/CSRF/company isolation, forged actor refusal, inert idempotent proposals, dual-grant self-review refusal, concurrent identical approval, version/stale rejection, prefix races, actual numbered draft → independent approval → balanced posting, publication versus first issuance, runtime SQL bypass refusal, forced apply rollback/retry and deliberately corrupted request/policy prior audit snapshots. Corrupted prior snapshots initially passed; migration 013 closes that gap and the tests now reject and roll back them.

The full backend suite currently has **110 passing tests**. `docs/browser-numbering-result.json` records **14 Chromium workflow checks**, including actual numbering and subsequent source cancellation without number reuse, independent revision/rejection, read-only/locked views, Escape, dark mode and 390px layout. Migration 012 implements configuration and the shared resolver; 013 strengthens prior-snapshot audit proof. Full existing regression/build/recovery artifacts should be refreshed with each later release.

Next dependency: central posting-rule configuration, followed by effective tax configuration and operational accounting/inventory/subledger integration. The tax calculation kernel alone does not authorize a rate or create a tax ledger.

Regression refresh

After migration 013, all nine browser suites passed in one sequential run (ending 2026-09-26T12:57:58Z). The numbering proof is NB27455100R-2029-00000001; its cancelled source retains its number and lock. Additional test-first regressions verify detail navigation during refresh and a numbering-only observer without finance grants. The current backend suite is 122 tests after adding 12 pure posting-rule selection tests. Optimized production-web smoke passed after both frontend fixes; this does not verify a production backend or Docker deployment.

Latest full regression refresh: all nine browser suites and optimized production-web smoke passed after the subsequent bootstrap/notification corrections; see per-suite result timestamps. Current numbering proof: NB29039616R-2032-00000001. Current backend: 123 tests. The numbering browser now includes the no-prior-user-notification assertion (15 checks).

Central posting rules — implementation contract

Status: **IN_PROGRESS**. Centrally reviewed account mapping now has a tested database/API/UI workflow; operational integration remains unfinished. Reuse the existing `accounting_rule_sets`, `accounting_rule_legs`, `journal_entries.rule_snapshot`, company configuration lock and `CommandService`. Do not create a parallel ledger or execute expressions supplied by administrators.

Workflow and boundary

1. An authorized human proposes an event/scope/effective-period/priority and symbolic-leg account mapping, with a reason. Pending proposals have no posting effect.
2. A different authorized reviewer applies or rejects the immutable proposal. Publication allocates its version under the exclusive company configuration lock. Replayed commands return the same result; stale or conflicting decisions roll back.
3. A trusted domain service computes named monetary totals from locked business records, obtains the effective reviewed rule under the posting transaction's shared configuration lock, validates account eligibility, and produces balanced journal lines.
4. Source, stock, subledgers, tax, journal, selected rule snapshot and audit commit atomically. Missing/ambiguous mapping or any subsequent failure rolls all effects back.
5. Posted entries retain their mapping/amount snapshots. Reversals use original effects; they do not reinterpret the original through today's mapping.

This does not retrofit automatic rules into the explicit non-control manual-journal workflow. No public operational posting endpoint is released by a resolver alone.

Deterministic selection contract (kernel first)

Input: company, event type, business date, branch and nullable warehouse/product/category/tax-category/payment-method IDs. Dates are real Gregorian dates in years 1–9999. IDs and input shapes are strictly validated. The caller must resolve and validate all entity relationships under company scope; matching UUIDs alone does not prove warehouse/branch or product/category ownership.

Candidates must be active, independently approved and effective on the business date. Effective endpoints are **inclusive**; a null end is unbounded. Company/event match exactly. Every non-null candidate selector must equal the corresponding context selector; null is a wildcard, not an unknown-value match. A constrained rule cannot match a missing context value.

Rank eligible candidates by **priority descending**, then **number of constrained selectors descending**. Exactly one highest-ranked candidate is required. No match fails closed. A tie fails closed even if one version/UUID is higher; version, approval time, input order and account IDs are not hidden tie-breakers. Invalid candidate structures/approval evidence are rejected rather than silently normalized into valid rules. A snapshot records the normalized context, selected immutable definition, creator/reviewer, version and ranking semantics; nested data must be immutable and detached from caller-owned objects.

This kernel selects a rule only. It is **not** proof of permissions, current account eligibility, approved event-contract membership, balanced line materialization, persistence, database immutability, or operational effects. Those gates remain mandatory.

Database and configuration requirements

- Reuse rule-set/leg tables. Add immutable proposal/decision records with tenant FKs, creator/editor/reviewer attribution and exact before/after audit proof. Active rule publication must require a matching independently applied request, not merely an `approved_by` UUID supplied in an `INSERT`.
- Append-only published mappings and leg definitions. Add explicit reviewed prospective retirement/effective-end records rather than overwriting historical mappings or choosing a higher version implicitly. Effective closure may not invalidate any already-posted rule application. Preserve original snapshots; fail visibly when a requested cutover is unsafe.
- Explicit registered event contracts define allowed symbolic legs, debit/credit direction, named trusted totals and eligible account/control categories. Account IDs, rates and account codes are never hard-coded into domain modules. Arbitrary formulas, eval and frontend totals are prohibited.
- Publication validates active same-company entities, hierarchy relationships, active/postable correctly denominated accounts and event-specific control-account eligibility. Reject possible equal-rank overlapping scopes/dates; the resolver still detects ambiguity defensively.
- New grants: `rules:view` / `rules:propose` / `rules:approve`. Role names confer no authority. Configuration review is independent; operational transaction approvals remain a separate policy.

- Planned API: paginated rule catalog/history, effective lookup, eligible account/scope search, proposal and decision. Strict request schemas, idempotency and optimistic versions; no direct counter/balance/account-classification edits.
- Frontend follows backend verification: permissioned Control center tab, server search, effective preview, inert proposals, independent review, immutable history, accessible dialogs and full state handling. A rules-only observer must be able to reach the tab without finance privileges.

Required verification sequence

Write pure selection tests before implementation: scope/tenant/event/date selection, priority/specificity, ties, missing rules, missing selector values, malformed/duplicate definitions, independent approval evidence, detached immutable snapshots and permutation invariance. Then PostgreSQL/HTTP tests for publication/retirement, tenant/RBAC/delegation, idempotency, audit corruption, locks, stale versions, account-policy validation, actual balanced domain posting and forced rollback. Only then frontend/E2E and module verification. No operational or production readiness claim is justified by the selection tests alone.

Current executed evidence

The complete backend suite now passes **185 tests**: resolver 12, materializer 10, overlap 7, registry 5 and configuration DB/HTTP 16 among the complete regression. Migration 014 is applied. Public configuration endpoints and the Control center tab support mappings, independent approval/rejection, lookup and prospective retirement. Exact audit, stale decisions, currency/scope eligibility, concurrent idempotency, rollback, forgery refusals and no financial side effects are covered. All 15 browser suites pass, including the new workflow and minimal rules-only access. Actual post014 backup restoration preserved publication links, 3 mappings, 6 requests and 3 retirements. These are configuration-only event contracts. Operational rule application, atomic application references, inventory/tax/subledger effects and reconciliation remain unfinished.

Trusted posting-line materialization

Status: VERIFIED for the internal trusted-input component (10 tests in the 146-test backend run). Operational integration remains IN_PROGRESS. This is an internal accounting dependency, not a public posting endpoint or evidence that operational modules are finished.

The resolver selects an independently reviewed effective mapping. Materialization must then validate it against a trusted, versioned event contract and current account records fetched inside the same business transaction. It accepts only named exact-decimal totals computed by the domain service, never frontend-calculated totals or executable formulas.

Required boundaries: - Exact event/contract version and complete symbolic leg set; mapping side/amount key must match the central contract. - Account IDs must resolve uniquely to active, postable, same-company accounts. The account classification/control type must match a contract eligibility pair. Null account currency means the company currency; otherwise denomination must match. - Only functional-currency posting is supported at this step. Foreign-currency conversion is not approximated or silently assumed. - Money stays decimal strings, with the same 16-integer-digit/two-fractional-digit per-line capacity as the journal API. Reject negative, floating-point, exponent or overprecision values. Reject missing/extra amount keys. - Omit zero legs, then reuse the existing balanced-journal validator. All-zero results are explicitly marked noFinancialEffect; the enclosing domain/source guards must authorize any zero-value operational posting. This helper does not provide that authorization. - Do not net same-account legs silently. Preserve symbolic legs and exact debit/credit effects. - Return a detached deeply frozen snapshot of the rule selection, registered contract, accounts, trusted normalized amounts and exact totals. Reversals will use posted snapshots, not reevaluate today's mapping.

Test first: correct controlled-account mapping, exact large cents, no binary numeric inputs, bad/missing/duplicate accounts, tenant/currency/active/postable/classification violations, altered leg semantics, unexpected totals, imbalance, explicit zero effects and immutable replayable snapshots. PostgreSQL publication, current-record locking, snapshot persistence, actor/RBAC enforcement and atomic inventory/tax/subledger posting remain separate mandatory integration work.

Executed evidence and boundaries

Missing-module RED preceded implementation; evidence is in `../testing/red-evidence/posting-materializer-red.log`. The helper reuses journalDraft money/currency schemas and validateLines; shared deepFreeze is in immutable.ts. Mapping version17 with event contract1 is explicitly tested. No public endpoint, record-loading/locking, persisted source snapshot or business zero-effect authorization is supplied by this helper.

Posting-rule publication — database/API implementation plan

Status: **VERIFIED** for company/branch/warehouse configuration. Operational application remains **IN_PROGRESS**. Migration 014 is applied and must not be edited. Database/HTTP RED preceded backend implementation; missing-tab browser RED preceded the UI. Current evidence: 185 backend tests (16 configuration tests), 15 browser suites including proposal → independent approval → lookup → retirement and isolated rules-only access, optimized production-web build/smoke, and actual encrypted backup restoration. See `../user-guide/posting-rules.md` and `../testing/posting-rules-restoration.json`. The sections below retain the design contract implemented in this slice, not a claim that inventory events now post.

Reuse and scope

Reuse `accounting_rule_sets`, `accounting_rule_legs`, `journal_entries.rule_snapshot`, existing account/organization tables, `CommandService`, configuration locks and immutable audit infrastructure. Manual journals remain explicit non-control entries. Rule configuration itself must never change a financial balance.

Initial centrally registered event contracts should cover the first inventory dependency: `GOODS_RECEIVED`, `STOCK_LOSS` and `STOCK_GAIN`. Register templates in reviewed migration-owned tables, not frontend constants or arbitrary administrator expressions. Mark them as configuration contracts, not released operational workflows. Do not expose receiving/adjustment posting merely because a mapping exists.

Event	Symbolic leg	Side	Trusted amount key	Required account eligibility
GOODS_RECEIVED	INVENTORY	DEBIT	stockValue	Active postable ASSET / INVENTORY control
GOODS_RECEIVED	GRNI	CREDIT	stockValue	Active postable LIABILITY / GRNI control
STOCK_LOSS	LOSS	DEBIT	stockValue	Active postable EXPENSE / non-control
STOCK_LOSS	INVENTORY	CREDIT	stockValue	Active postable ASSET / INVENTORY control
STOCK_GAIN	INVENTORY	DEBIT	stockValue	Active postable ASSET / INVENTORY control
STOCK_GAIN	GAIN	CREDIT	stockValue	Active postable REVENUE or EXPENSE / non-control

Account IDs/codes are company configuration, never constants in a domain module. `stockValue` will come from trusted receiving/valuation calculations, not a frontend total. Zero-value movement semantics require an explicit no-financial-effect contract in the inventory/source guards; do not invent a zero-sided journal or claim that the current source posting guard already supports it.

Planned relational additions

- Migration-owned event contract/version and leg-contract tables: event, contract version, leg code, side, named amount key and eligible account classifications. Normalize eligible classification pairs rather than placing executable formulas in JSON. Runtime read-only. Record migration origin instead of inventing a human or authentication-service creator.
- `posting_rule_change_requests`: company/creator/time, kind `CREATE` or `RETIRE`, immutable reason/payload, `PENDING/APPLIED/REJECTED`, independent reviewer/time/reason, applied resource link. Tenant-composite FKs for every scope/reference. Generated intended new rule ID is an immutable reservation, not a counter or an already-existing FK target.
- `posting_rule_change_legs`: normalized immutable proposed leg/account pairs with company/request/account FKs and unique symbolic leg. `CREATE` proposals supply only `legCode/accountId`; `side/amountKey` come from the versioned registered contract. Require the complete exact leg set.
- Extend existing rule sets with registered contract version and publication-request linkage. Preserve their existing company/event/version uniqueness. Add any needed composite uniqueness for child contract FKs. Existing inactive owner-created test/domain placeholders must not be mistaken for reviewed active rules. Inspect existing fixtures before tightening `NOT NULL/FK` constraints; never erase rows to make a migration pass.
- `accounting_rule_retirements`: append-only reviewed effective-through records and per-rule retirement version. Each subsequent retirement may only shorten effective coverage, never edit a prior record or resurrect an old mapping. Store creator/reviewer/request linkage. Effective end is the minimum of the original `valid_to` and reviewed retirements.
- Normalized journal-rule application references will be required when domain posting is integrated, so retirement can check already-posted application dates without trusting caller JSON. Until then do not claim operational snapshot persistence is complete.

Published rule definitions/legs are immutable. Retirement changes effective selection by adding a separately reviewed record, not rewriting a historical mapping. A replacement is a new version with explicit non-overlapping dates. There is no hidden latest-version tie-breaker.

CREATE publication

1. Require rules:propose; validate event contract, dates, selector shapes, active same-company scope entities and hierarchy relationships, complete legs and eligible accounts. Store immutable proposal/legs and exact audit under CommandService's exclusive company configuration lock. Pending requests produce no rule or journal.
 2. Approval requires rules:approve and a different human from the proposer, even if one person holds both grants. Rejection also retains separate review attribution. Mandatory independent review is a safety minimum, not a claim that all future multi-stage configuration approval policies are finished.
 3. Recheck current authorization, membership, scope hierarchy, accounts, event contract and possible equal-rank effective overlaps. Acquire the exclusive company lock before resource locks; avoid a second numbering/locking engine.
 4. Allocate the next company/event version under that lock. CREATE has no mutable target version; uniqueness and conflict checks handle intervening publications. RETIRE must compare its captured retirement version.
 5. Insert the immutable rule/legs retaining the proposal creator; reviewer and publication time remain separate. Link terminal request and exact deferred before/after audit. Any failure rolls back all changes and the command result.
 6. Reject potential equal-priority/equal-specificity overlaps when all constrained dimensions are compatible and date intervals intersect. Conservative publication rejection is acceptable even if a currently higher-priority rule masks the conflict. Resolver ambiguity remains a defensive failure, not an ID/version fallback.
-

RETIRE publication

Require a target active reviewed rule, expected current retirement version, inclusive effective-through date and reason. End must be within the rule's coverage, not earlier than its start or the current Nairobi date, and must not invalidate an already-posted application. A later retirement may only shorten the current end; extension requires a new rule version with explicit coverage. An already-posted original remains historical evidence even if reversed; conservative retirement refusal is preferable to silently changing its eligibility. Document this restriction visibly.

A finite higher-priority rule can expire and reveal a distinct lower-ranked fallback; this is explicit priority behavior. Retired versions must not silently resurrect. At effective lookup, apply retirement intervals before invoking the tested pure resolver; the snapshot must identify the applied retirement evidence as well as the immutable base definition.

Database defense and audit

Runtime must not publish by supplying approved_by alone, insert unmatched legs, change a published definition, forge creator/reviewer, move scope/ownership or rewrite a terminal request. Deferred guards require the exact approved request and exact normalized leg set. Validate audit actor, request ID, before/null snapshot and after snapshots; include negative tests for forged prior snapshots, following the numbering 013 lesson. Role names and frontend actor IDs never confer authority. The trusted owner remains a migration/recovery boundary, not a runtime login.

Treat the existing active flag carefully: publication/retirement must not mutate old definitions just to hide overlaps. Selection must use reviewed retirement records and historical snapshots. Unreviewed/inactive legacy fixture rows cannot be selected or exposed as published rules.

Planned API and UX

Grants: rules:view, rules:propose, rules:approve. Planned routes: GET /admin/posting-rules, /contracts, /effective, /accounts, /scope-options; GET/POST /admin/posting-rule-changes; POST /admin/posting-rule-changes/:id/decide. Use bounded server pagination/search and strict schemas. Dedicated eligible-account search is authorized by rules:view and reveals configuration, not ledger balances; it must not require or imply accounting:view.

Catalog/effective responses must distinguish no match, ambiguity, pending proposals, published versions and effective retirement. A rules-only observer must reach the Control center tab. Account/branch labels are presentation metadata; selection uses validated IDs. Frontend follows backend tests and must show maker/reviewer, immutable history, explicit effective dates, conflicts, read-only/empty/loading/error/success states, keyboard/focus, mobile and dark mode.

Next test-first sequence

1. PostgreSQL/HTTP fixture with cleanup covering setup/bootstrap failures; initial missing-route red evidence.
2. Proposal inertness, strict input, complete leg contracts, tenant/hierarchy/account policy checks, grants/CSRF and independent review.
3. Idempotent concurrent decisions, version allocation, compatible-overlap races, exact before/after audit corruption and direct runtime tampering.
4. Effective query versus the pure resolver, explicit prospective retirement, stale retirement decisions, rollback and no silent fallback/resurrection.
5. Trusted exact-decimal line materialization and frozen snapshots, then actual runtime-role source/journal/domain integration with intentional rollback.
Do not substitute owner-only SQL or manual journal coincidence for evidence that automatic posting rules are enforced.
6. API docs, frontend, E2E, visual inspection and status update. Keep the overall Accounting rules requirement IN_PROGRESS while operational application persistence and financial integration remain incomplete.

Use a draft migration location outside the live root migration manifest while the SQL is under development, or clearly account for readiness becoming false when a root migration is pending. Move/apply the numbered migration only after tests and an encrypted pre-migration backup. Never alter applied 001–013 checksums.

Publication overlap predicate (29 September dependency)

Before database publication, add a pure `postingRuleScopesConflict` predicate using the resolver's same selector/date/priority schemas. Inputs are applicability scopes only, not approvals or account bindings; the caller must load approved published candidates and apply reviewed retirement ends under the configuration lock. A conflict exists when company/event match, priority and specificity tie, inclusive intervals intersect, and no dimension has contradictory non-null selectors. Different contract/mapping versions do not resolve a tie. Cross-dimension hierarchy contradictions are deliberately not used to waive conflicts: conservative rejection remains safe when masters later change. Test every dimension, inclusive/open dates, invalid inputs and exhaustive finite contexts against resolver behavior. This predicate does not implement database locking, authorization, publication or retirement.

Migration-owned registry sealing

The staged 014 registry uses normalized contracts, symbolic legs and eligibility pairs. Each definition is inserted unsealed, populated and sealed in one owner migration transaction. A deferred guard requires 2–100 legs, both debit/credit sides and 1–20 eligibility pairs per leg. Headers cannot change except the one-way seal; legs/eligibility cannot be updated/deleted or appended after sealing. Runtime has `SELECT` only (no insert/update/delete/truncate); migration principal/time are recorded from the actual PostgreSQL role/transaction, not a fabricated human. Workflow status remains `CONFIGURATION_ONLY`. Draft SQL is exercised only in disposable database tests until publication guards are complete and 014 is promoted through a pre-migration backup. The live 001–013 manifest/schema remains unchanged meanwhile.

First released configuration surface

Implement company-wide, branch and warehouse scopes first; product/category/tax/payment selectors remain internal resolver capabilities, not accepted proposal fields until their masters are released. Warehouse proposals require a matching active branch. Configuration permission is company-wide (as with numbering), not a grant to view/post branch transactions. Supported currency is the company functional currency; current account eligibility is rechecked at approval. There is no balance/stock/tax effect from configuration.

Release `CREATE` and prospective `RETIRE` through immutable normalized requests, independently reviewed decisions, exact SQL-native audit snapshots and shared company configuration locking. Preserve immutable base rules, append retirement versions, reject overlapping equal ranks and stale retirement requests. A normalized journal-application reference table remains runtime read-only until operational posting is implemented; retirement checks any retained application dates. No receiving or stock adjustment endpoint is enabled by configuration. Frontend includes contract/account/scope search, pending review, applied history, effective lookup, rejection/retirement, read-only access, keyboard dialogs and responsive states. Other four selector dimensions remain visibly unavailable rather than pretending to be supported.

Sandbox recovery and continuity — 29 September 2026

Status: IN_PROGRESS. Local encrypted recovery is tested; production disaster recovery and latest-data continuity are not verified.

What happened

A sandbox reset removed dependencies, processes and the native PostgreSQL cluster in `.cache/postgres`. Source files and private encrypted backups survived. An available authenticated archive was restored into the new isolated `erp_restore_sandbox_20260929` database and verified before promotion. The `erp` target was confirmed absent; no existing live database was overwritten. The restored snapshot held 13 source documents, 11 journals, 22 lines and 173 audit events; journal arithmetic passed. Receipt: `../testing/sandbox-restoration-20260929.json`.

That archive predated migrations 012–013 and later demo activity. The unchanged migrations and demo grants were reapplied without silently resetting existing passwords. Later original test history was not recoverable from this snapshot. Subsequent browser runs created new explicitly demo-only transactions; these are not invented reconstructions of missing transactions.

After the latest nine-browser regression chain, a new encrypted checkpoint was taken at 2026-09-29T05:31:49.217Z: 19 sources, 15 journals, 30 lines, 269 audit events. The private `.runtime/latest-backup.json` identifies the archive and manifest; use the newest valid authenticated checkpoint on another reset, not the `pre012` archive by default. A separate current six-test recovery suite exercises real isolated DB/file restoration, consistent counts, journal arithmetic, runtime protection and refusal of corrupt/unsafe targets. It does not establish every operational reconciliation.

Production write reopening is a separate gate

Stale recovery can rewind counters and lose recently committed transactions, external payment references, idempotency keys and review history. A balanced restored journal proves arithmetic, not completeness. Allocator uniqueness inside one surviving database does not establish global nonreuse after lost history.

Before production writes resume: 1. Preserve surviving data/logs and establish a reviewed recovery point/RPO. 2. Prefer tested continuous WAL/PITR plus off-host archives and independent key recovery where the business requires a low RPO. 3. Reconcile issued document ranges against surviving fiscal/receipt records, branches and any external issuance authority. Establish a reviewed safe series/cutover through controlled migration/configuration; never invent missing transactions or arbitrarily jump counters to conceal uncertainty. 4. Reconcile external payment/provider references and delivery outcomes before retrying uncertain commands; an STK request is not payment proof. 5. Reconcile every implemented ledger/control balance, preserve visible differences, and obtain authorized cutover approval before reopening writes.

These production gates remain unfinished. No silent balance adjustments or counter repairs were made during this sandbox recovery.

Development limitations

The native cluster is cached and not durable across Arena workspace resets. A manual encrypted checkpoint protects data only through its snapshot time. Interactive activity after that checkpoint can be lost if no newer backup is made. There is no claimed background checkpoint scheduler or production volume/WAL persistence here. Do not use this sandbox for real financial operations. Production persistence/reboot/container-recreation tests require the intended host and remain blocked/unverified.

Tax calculation kernel — dependency work, not a released tax module

The existing journal/approval engine remains the posting authority. This work adds a pure exact-decimal calculation component while configuration/numbering and operational services are still being completed. It does not expose an endpoint accepting client-selected tax rates, seed legal rates, create tax ledger rows or claim eTIMS readiness.

Inputs: strict decimal strings for quantity/unit price/discount; business date; company currency; explicit category classification; authoritative effective-dated rate snapshots now supplied by the permission-checked informational configuration preview; operational resolution remains unfinished. Rates are fractional factors (e.g. 0.16 in tests), not hard-coded jurisdiction constants. Current ledger scale is two decimal places, matching numeric(20,2); unsupported scales fail rather than silently truncating. Products/quantities/unit prices may have six decimal places, rates/recoverability ten. No JavaScript floating-point money arithmetic.

For TAXABLE, require at least one positive configured rate. ZERO_RATED requires explicit zero-rate snapshots; EXEMPT and NON_TAXABLE require no applied rates and preserve their distinct classification. Duplicate rate/tax identifiers and out-of-date snapshots fail. All components use the same net base; compound taxes are explicitly unsupported until a reviewed rule model is implemented. All applied snapshots must agree on inclusive/exclusive mode. Discount is a currency amount applied to the rounded line extension, before tax, and cannot exceed that extension.

Exclusive: round line extension, deduct discount, compute each component from net and round half-up to cents; gross = net + component taxes. Inclusive: rounded discounted extension is gross; compute unrounded net by dividing by one plus the sum of rates, then round net half-up. Tax = gross – net. Allocate that exact tax total using largest remainders over each unrounded component, with stable tax-ID/rate-ID tie-breaking; never leave a hidden discrepancy. Recoverable tax is rounded from component tax × configured recoverable fraction; nonrecoverable tax is the explicit remainder.

Output includes engine version, rounding/allocation policy, original normalized quantity/unit price/discount, full effective rate IDs/dates/factors/recoverability, classification, currency, price mode, net/tax/gross, component breakdown and recoverable/nonrecoverable amounts. Returned snapshot and nested structures are frozen. Serializing this snapshot must retain enough information to reproduce the calculation; later rate changes cannot alter it. Reject final values outside the current numeric(20,2) range.

Tests cover inclusive/exclusive, zero/exempt/non-taxable distinction, partial recoverability, effective boundaries, invalid dates/rates/float input/discount/duplicate IDs, rounding residual distribution, order-independent calculations, snapshot replay, and many small-value/rate combinations proving $\text{net} + \text{tax} = \text{gross}$ and $\text{recoverable} + \text{nonrecoverable} = \text{tax}$. Finite-schedule database configuration, independent approval and ID-based preview now also have database/HTTP/browser evidence; see tax-configuration-next.md. Tax registers, retirement/amendment, return allocation and operational accounting integration remain unfinished, not verified on the strength of calculator tests.

Controlled tax configuration — next dependency

Status: IN_PROGRESS overall. The initial finite-schedule configuration slice is VERIFIED through database/API/UI, independent review, exact audit, ID-based calculation and isolated restoration. Migration015 is applied; never edit it. Current backend185 passes, including14 tax DB/HTTP tests. New browser-tax and minimal tax-only access suites pass alongside all11 prior browser suites. **Retirement/amendment, product assignment, tax transaction registers and operational posting remain unfinished.**

Implementation decision: retain the existing PostgreSQL tax/category date exclusion unchanged. Require explicit finite inclusive dates; published schedules cannot yet be shortened. Adjacent schedules are supported, overlapping schedules rejected. This deliberately avoids falsely implementing append-only retirement while the old exclusion still covers the original interval. No active statutory rate is seeded; browser-created DEMO schedules are calculation-only test configuration. The remaining design below includes future capabilities, not claims that all are released.

Reuse and explicit boundary

Reuse taxes, tax_categories, tax_rates, tax_transactions, accounts, CommandService, company configuration locks and calculateTaxLine. Do not create a parallel calculator or ledger. No hard-coded statutory rates, active demo VAT schedule, caller-supplied actor IDs, arbitrary formulas or fake eTIMS certification. Configuration writes never create a journal, tax transaction, stock movement or balance.

Migration 015 or later must be additive: applied001–014 are immutable. Inventory/POS remain disabled until authoritative selection, tax snapshots and atomic posting are integrated. Names/types/categories are configuration, not legal verification of a taxpayer or tax treatment.

Workflow and normalized state

1. Authorized human proposes a tax definition, classified category, or effective rate/account binding with a reason and documentary basis. Preserve proposed values in typed normalized columns; do not hide financial configuration in free-form JSON.
2. Pending proposals are inert. A different authorized human approves/rejects; creator/editor/reviewer/time remain distinct. Approved definition identities and classification are immutable. Naming/lifecycle changes need explicitly reviewed amendments rather than direct updates.
3. Approved schedules bind tax + category + inclusive dates + exact fractional rate + inclusive/exclusive mode + recoverability + input/output account IDs. No priority-based silent tie breaking: the existing tax-rate exclusion prohibits overlapping schedules for a tax/category; retain deterministic fail-closed semantics.
4. Replacement/retirement must preserve published definitions and historical snapshots. **Resolve the existing base-table exclusion before choosing append-only retirement:** a valid_to=9999-12-31 row continues to overlap even when a separate retirement record shortens its effective coverage. Do not remove the exclusion without replacing its concurrency/integrity guarantees. Tests must establish immutable history and race-safe effective intervals first.
5. Runtime selection uses server company, transaction date and authoritative category/tax assignments; returns exactly one applicable schedule per required tax or fails. Never trust the calculation preview as posting authority. Future operations persist selected rate IDs, effective-end revision, normalized exact factors, account bindings, classification, price mode, calculation version and full results inside their business transaction.
6. Returns/credit notes use the original persisted tax breakdown with cumulative limits, not current configuration. Corrections to prior wrong tax require linked business adjustments; never rewrite posted snapshots or silently revise ledgers.

Validation / permissions / attribution

Use separate taxes:view, taxes:propose, taxes:approve permissions, no role-name authority or implicit finance read grant. Use backend RBAC plus tenant FKs/RLS, strict schemas, server actors, idempotency, serialized configuration publication, immutable terminal decisions and exact SQL-native audit snapshots.

TAXABLE requires an applicable positive configured factor; ZERO_RATED requires an explicit zero schedule. EXEMPT and NON_TAXABLE remain distinct and apply no rates. A bounded same-base multi-tax group must agree on inclusive/exclusive mode; compound rules stay unsupported. Decimal-string factors/recoverability only; no JS floating-point money. Gregorian dates must also fit PostgreSQL (no year0000), and end dates are inclusive.

Revalidate active/postable company accounts, functional denomination and required tax control/class at approval and posting. Establish the actual control types from current account schema/seed rather than inventing account numbers. Input recoverability is not blanket legal entitlement:

product/use/document/period eligibility still belongs to the later operational tax policy. Initially refuse unsupported configuration combinations rather than inventing automatic tax postings.

API/UI shape to prove, not yet released

- Bounded searchable catalogs for taxes/categories/rates, eligible accounts and request history.
- Strict proposal endpoint with discriminated typed changes; independent decision endpoint.
- Effective lookup/calculation preview accepts configuration IDs and date/quantity/price/discount, never authoritative raw factors/accounts/actors. Clearly marked informational, with no journal/stock/tax-register effect.
- Reuse ConfigurationPicker and control-center dialog/focus/error/loading/empty/confirmation patterns after backend verification. Expose classification and both rate factor/percentage display without ambiguous conversions. Show basis, effective dates, account names, creator/reviewer and retirement/replacement history.

Test-first gates

Write isolated PostgreSQL/HTTP RED tests before migration/service: strict unknown-field rejection, unauthenticated/RBAC/tenant isolation, no implicit finance access, exact decimal boundaries, leap/invalid dates, zero/exempt distinction, malformed/account currency/control/scope checks, inert proposals, self-review denial, immutable decisions/definitions, duplicate code and interval races, idempotency/retry, stale retirement, rollback after partial publication, exact audit/forged-audit refusal and absence of financial side effects. Include open-ended schedule replacement and retained-application-date protection before enabling retirement.

Then real browser propose → review → effective preview, minimal tax-only role, paginated account search, dark/mobile/focus checks and existing regressions. Fresh encrypted checkpoint and actual isolated restore must preserve approved schedules, requests, audits and relationships. Only then wire trusted configuration into product assignment and atomic operational transactions. Full tax registers, returns, tax-account reconciliation, eTIMS and legal acceptance remain separate gates.

Legal reference boundary

Official KRA VAT information reviewed earlier: <https://www.kra.go.ke/individual/filing-paying/types-of-taxes/value-added-tax> . Preserve documentary effective dates and obtain current authorized review before activating rates. General/zero/exempt classifications and tax-point/recoverability rules must not be reduced to an unconditional percentage. Current contracted eTIMS specifications and external acceptance remain required; no fiscal success is simulated as production evidence.

Product reference masters — first inventory dependency

Status: VERIFIED for initial immutable creation/read/search; overall inventory IN_PROGRESS. Migration016 applied; nine DB/HTTP tests, workflow and isolated product-only browser access tests pass. Actual restoration preserved metadata and exact audit links. Lifecycle/corrections remain unfinished. This slice implements real units, product-category hierarchy and brands before product/SKU and movement services. It is not stock CRUD.

Workflow and effects

An authenticated company member with `products:manage` creates a unit, category or brand with a reason. `products:view` reads/searches bounded catalogs without receiving ledger access. Reuse `CommandService` for session/permission revalidation, company configuration locking, idempotency, rollback and failed-attempt audit. Creation of reference metadata has no financial, inventory, tax, quantity or balance effect and does not require a monetary approval threshold. Future product/tax/price changes need their own explicit workflow; this permission does not authorize them.

Reuse `units_of_measure`, `product_categories`, `brands`. Add an explicit catalog version marker so legacy owner-created target-schema placeholders are not silently adopted. New governed records are immutable: no editing unit precision or category parent after use, no delete/recreate. Root/child category creation must stay in the company, reference governed parents, avoid cycles and bound hierarchy depth. Units have an integer quantity scale0–6; brands have names; units/categories have unique uppercase codes. Caller actor IDs and unknown properties are rejected. Preserve the actual creator and creation time and exact SQL-native audit in the same transaction. Published reference metadata cannot be modified even by ordinary owner statements; future lifecycle migration must preserve history.

API and UI contract

GET `/api/v1/admin/product-references?kind=UNIT|CATEGORY|BRAND&q=&page=&limit=`; POST on the same path accepts discriminated typed creation plus reason. IDs are server generated. The Control center Product setup tab exposes separate catalogs and a creation dialog, keyboard-focus handling and category-parent server search. No stock quantities, prices, ledger accounts or tax percentages are editable here.

Test-first gates

HTTP404 RED before implementation. Then real DB/HTTP tests for permission/tenant/CSRF/strict schema, idempotency, duplicate code/name races, inactive authority, unit scale bounds, hierarchical parent scope/depth, exact immutable audit and deliberate audit corruption rollback. Assert no product, source, journal or movement is created. Run existing regression, then browser creation/catalog/search, minimal reader, mobile/dark and focus. Backup/restoration must preserve metadata and audit. Only then start full product masters with SKU uniqueness, barcodes, reviewed tax references and exact prices; movement-only weighted-average inventory remains a separate atomic operation, not a field on the catalog editor.

Reviewed product/SKU registration and assignment boundaries

Status: VERIFIED for independently reviewed initial registration only. Product lifecycle, price/profile amendments and inventory operations remain IN_PROGRESS or NOT_STARTED in the live matrix. Reuse the verified references and existing engines.

Reuse and sequence

Reuse governed units/categories/brands (migration016), reviewed tax masters/schedules (015), existing products, product_barcodes, product_taxes, product_prices, money/quantity domains and CommandService. Design normalized versioned changes before coding. Continue with product creation/read/search, controlled metadata/pricing/tax-assignment changes and barcode uniqueness; only then movement-only inventory and weighted-average costing. Do not expose quantity-on-hand or derived stock valuation as editable master fields.

Decisions that must be explicit

- SKU and product-code uniqueness and normalization, barcode text preserving leading zeros, exact-match barcode fast lookup plus bounded name/SKU search. Concurrent creation must not duplicate identities.
 - Required governed category and base unit, optional governed brand. Supplier dependency remains separate until supplier masters exist. Legacy owner-created placeholders must not silently become operational products.
 - STOCK/SERVICE/NON_STOCK semantics. Batch/serial/expiry options must be rejected or visibly unavailable until their operational invariants exist; serialized quantities must respect indivisible units.
 - Product prices use exact six-place strings compatible with numeric(20,6), with explicit currency and price mode policy. Separate commercial reference prices from inventory valuation. **Do not round-trip SQL JSON numeric prices through JavaScript Number**:20 significant digits can lose precision. Use SQL-native audit assembly or canonical decimal-string snapshots, and test the maximum supported value.
 - Product category classification and tax IDs need dated authoritative assignments. A mutable current products.tax_category_id alone cannot resolve historical tax correctly. Design effective-dated classification/assignment history, uniqueness/overlap and independent review before allowing taxable products to become operational. Preview must not authorize posting; missing applicable schedules fail closed.
 - Product/price/tax changes preserve original actor, reviewer where required, time, reason and immutable history; optimistic versions prevent lost updates. Original posted sale/return snapshots must not reinterpret current product metadata, rates or prices.
 - Create product, barcode/assignment children, exact audit and idempotency result atomically. Deliberate child/audit failures must roll back every local effect. No opening stock, journal, customer/supplier balance or tax register is created by master-data registration.
-

Original test-first plan (executed for initial registration)

Add isolated DB/HTTP acceptance tests before migration017/service: authentication/RBAC/tenant/strict body, foreign or legacy references, exact quantity/price bounds, forbidden balance/actor fields, duplicate SKU/code/barcode races, leading-zero barcode preservation, coherent dated tax assignment, inactive account/reference revalidation where applicable, immutable/optimistic changes, exact large-decimal audit and rollback/retry. Reuse existing engines rather than introducing another ledger/tax calculator. Verify backend before adding the product screen; then real browser creation/search/barcode/least-privilege/mobile/dark and full regression. Fresh encrypted backup and actual restoration are required before marking the slice verified.

Current slice design decision — 29 September

Implement independently reviewed **initial registration**, not direct product mutation. PENDING → APPLIED/REJECTED requests store typed initial prices, references, a finite initial tax profile and normalized proposed tax/barcode children. All publications occur atomically in the independent review transaction. Use products:propose/approve separately from reference-management authority; approving initial prices also requires products:cost:view. Ordinary products:view catalogs/history mask purchase cost. Seeded administration/reviewer receive explicit grants; no role-name checks.

Products retain immutable initial commercial prices in company functional currency and explicit INCLUSIVE/EXCLUSIVE mode. A separate immutable finite `product_tax_profiles` record captures dated classification; existing `product_taxes` rows link tax IDs to that profile and its exact interval. Do not resolve from the current header alone. Approval requires full interval coverage by reviewed compatible same-base rate schedules, not merely a rate on the first day. Exempt/non-taxable profiles require no taxes and exclusive mode. Dated profile amendment, commercial price revisions and operational tax posting remain subsequent work; never create an overlapping replacement silently.

Initial barcodes refer only to the base UOM and remain text. Governed products/children cannot be mutated or extended after publication. SKU/product codes use explicit uppercase ASCII identity policy; printable no-space barcodes retain case and leading zeros. Batch/serial/expiry, alternative UOMs, supplier linkage, reorder/balance fields and operational posting are not accepted in this registration body.

Add product catalog/detail/search, exact barcode lookup, request history, proposal/decision and a server-price tax preview. Reuse the configured tax calculation path inside the same read transaction; do not create another calculator or forward frontend factors. Preview is informational and fails outside the retained profile/schedules. Exact numeric(20,6) prices travel as strings; SQL assembles audit snapshots without JavaScript numeric round trips. Existing target-schema legacy products stay excluded from governed catalogs.

Executed evidence and limits

Migration017, `product-configuration.ts` and `Products.tsx` implement this initial contract. Backend product suite:17 tests including parent, within214 passing backend tests. Real browser registration/review/search/barcode/preview and restricted-reader-with-audit suites pass, alongside all15 earlier browser suites. Maximum9999999999999999.999999 prices survive native SQL audits and authorized API audit projections exactly. Product audit snapshots are redacted without current `products:cost:view`; generic audit authority alone cannot bypass cost visibility. Original database audits remain intact.

Inspection found and fixed audit-route cost exposure and JSON-number precision loss; expected RED and final GREEN evidence are retained. Mobile/dark screenshots were inspected; dialog errors now clear on dismissal instead of leaking into the catalog. SQL publication, duplicate barcode races, schedule gaps, stale tax accounts and deliberate audit/child failures are covered. No stock or financial effects arise from registration. Actual encrypted restoration: `../testing/product-registration-restoration.json` (17 migrations, two registered products, four barcodes, exact native publication audits, zero stock movements).

Seven routes across six OpenAPI paths expose catalog/detail/lookup, request list/propose/decide and server-price preview. UI: Control center → Products. Initial registration is immutable; there is no product/price/profile correction or retirement command, supplier linkage, alternate-unit barcode, stock availability, valuation posting or sales flow. Continue with the stock transaction contract in `weighted-average-contract.md`; do not mistake its arithmetic kernel for movement posting.

Moving weighted-average inventory: contract before implementation

Status: IN_PROGRESS. The initial arithmetic kernel is VERIFIED within its pure scope; **no inventory API, stock posting or balance-editing screen exists**. This document is not operational verification.

Reuse, entities and authority

Reuse products and governed base units (017/016), source_documents, inventory_movements, the rebuildable inventory_balances projection, existing workflow/CommandService, reviewed posting-rule resolver/materializer and immutable journal engine. Do not introduce another ledger or tax calculator. Product purchase reference price is **not** authoritative valuation or an opening balance.

A stock pool is (company, warehouse, product), initially STOCK items in their governed base unit. Batch/serial/expiry, alternate units, FX, landed-cost allocation and reservations need separate contracts before activation. Pure valuation receives **trusted** normalized values from a future domain transaction, never HTTP authority.

Arithmetic decisions

- Exact decimal strings only. Quantity and unit-cost domains are numeric(20,6); monetary value is numeric(20,2). Accept up to 14 integer and 6 fractional quantity digits, and 18 integer and 2 fractional money digits. Enforce configured unit precision 0–6. Reject exponent notation, negative values, floats, empty/overprecision values and unknown keys.
 - State contains nonnegative base quantity Q and functional-currency value V . $Q=0$ requires $V=0$. Positive zero-valued stock is mathematically valid, but this does **not** authorize a zero-value operational source.
 - RECEIVE trusted positive q and nonnegative cent value v : $Q'=Q+q$, $V'=V+v$. v must eventually come from approved receipt/adjustment logic, not a supplied product cost or editable balance.
 - ISSUE positive $q \leq Q$: if $q=Q$, $\text{cost}=V$; otherwise $\text{cost}=\text{HALF_UP}(V \times q / Q, 2)$. Compute directly from full exact state; **never multiply by a rounded six-place average**. $Q'=Q-q$, $V'=V-\text{cost}$. Quantity/value deltas are negative for an issue.
 - Last depletion consumes every remaining cent. Earlier rounding remains visible in immutable movement values; no invisible corrective journal or orphan zero-quantity value is permitted.
 - Six-place movement unit cost is $\text{abs}(\text{value delta})/q$, rounded HALF_UP, for attribution/display. The authoritative monetary effect is the signed cent value, not unit cost $\times$ quantity. Derived average costs are informational only. Reject results outside current domain capacities, including unit cost; do not silently cap.
 - Functional currency is explicit ISO uppercase code, scale 2 only. No conversion and no hardcoded KES rate. Return a detached deeply frozen versioned input/output snapshot. Use a local Decimal constructor so global accounting rounding is unchanged.
 - The first helper supports RECEIVE and ISSUE only. Returns, reversals and supplier returns must use retained original movement cost and cumulative limits, **not** an ISSUE at today's average. Transfers must conserve exact source value between both locked pools; cross-branch rules and in-transit ownership remain a separate workflow.
-

Future transactional workflow (NOT_STARTED)

- Attributed DRAFT source with stock lines, reason, warehouse, product and effective date; submission snapshots reviewed policy. No stock effects in DRAFT/PENDING_APPROVAL. Independent configured approval is required before posting. Retain creator/editor/approver/poster/reverser separately; system actions retain original human initiator.
- Posting authenticates granular event authority and scope, checks source revision/state/period, governed product and live warehouse. Never trust supplied actors, balances, aggregate values, averages or posting accounts. Prices/costs require their own view/proposal/approval authorities; no role-name checks.
- Serialize a deterministic sorted set of company/warehouse/product pool locks, including creation of absent projections; re-read source/stock/configuration under lock. Idempotency keys and movement uniqueness suppress duplicates. Forbid negative stock under concurrency. Reject dates earlier than the latest valued date until an explicit recalculation policy exists; same-day posting order is retained by projection version.

4. Domain service computes trusted receipt value or issue cost with this helper, resolves current approved symbolic mappings, materializes balanced lines through the existing engine, and persists the source snapshot, immutable movements, derived projection version, journal, attribution, audit and idempotency result **in one local transaction**. Any child/audit/journal failure rolls back everything.
5. No generic direct balance CRUD. Database guards must bind projection deltas to complete immutable movement sets and bind stock-control journal value to the same signed cent effect. Runtime grants remain absent until these guards/tests exist. A future all-zero financial effect requires a deliberate source contract: `existing inventory_movements.journal_id NOT NULL` must not be bypassed with a fake zero journal.
6. Reversals are new linked approved sources with original amounts, cumulative quantity/value bounds and independent traceability. Never mutate/delete posted stock or GL history. For unavailable returned/reversed stock, fail visibly rather than silently creating negative inventory.
7. Reconciliation recomputes balances from movements and matches valuation to stock-control GL, with visible discrepancies and drilldown. Operational stock, COGS and inventory reports are not released until that reconciliation passes.

Planned API and UI follow database/HTTP acceptance: source draft/submit/review/post/reverse endpoints; read-only stock search/movement drilldown; governed pickers, exact quantities and costs; no quantity-on-hand form. Endpoint names, initial event contracts and permissions must be finalized with the first source workflow rather than advertising nonexistent routes.

Acceptance order

First pure tests: receipt/issue weighted average, full depletion, fractional-unit precision, large exact cents, zero-value state, overflow/negative-stock rejection, strict schemas, detached immutable replay and arithmetic matrices. Then isolated PostgreSQL/HTTP tests **before** migration018: permissions/tenant/branch, current reference/cost-policy checks, conflicting dates, projection creation and issue races, idempotency, exact native snapshots, source/movement/GL/audit coupling, forged SQL refusal, forced rollback and reconciliation. Only then implement a stock UI and real browser scenarios. Fresh backup plus isolated restore remain mandatory.

Executed kernel evidence — 29 September 2026

Test-first missing-module RED preceded `apps/api/src/inventory-valuation.ts`. Twelve tests now pass, including 3,000 full depletion states (46,500 issue steps), exact large cents, precision/capacity/negative-stock refusals, retained cent residual, no rounded-average multiplication, immutable replay and unchanged global Decimal settings. The first freeze test incorrectly expected sloppy-mode assignment to throw; corrected it to assert `Reflect.set` fails and the frozen value remains unchanged. Full backend suite: 214 passing tests, 73 top-level, 42,442.863354ms. Evidence: `docs/testing/inventory-valuation-results.txt`, `docs/test-results.txt` and `docs/testing/red-evidence/inventory-valuation.txt`.

No PostgreSQL movement writes, stock API, UI, source authorization, idempotency/concurrency locks, original-cost returns, reconciliation or operational journal binding are supplied by this helper. Existing movement `journal_id NOT NULL` and the journal materializer's narrower per-line input capacity remain explicit integration gates. Next: finalize the first stock source contract and write isolated database/HTTP failures before any 018 migration or operational endpoint.

Reviewed stock adjustment and full-reversal contract (018)

Status: VERIFIED for reviewed single-product gains/losses/full reversals only. Inventory overall remains IN_PROGRESS. Migration018 is applied and immutable (019 customer registration is also applied). Backend, extended workflow/access browser and actual encrypted018/019 restoration pass. All21 runtime/UI browser suites passed before the later unreferenced credit helper.

Initial workflow: a single STOCK product/base-unit line in one active scoped warehouse. This is a reviewed stock adjustment, not a purchase/receipt, sale, tax-calculation workflow, transfer or opening-balance import. STOCK_GAIN records proposed positive quantity and a documented positive total functional-currency value; that value is a proposal requiring review, not posting authority. STOCK_LOSS accepts quantity only and derives its cent value from locked stock. Neither accepts a balance, unit cost, account, actor or tax amount. Zero-cent adjustments are refused until zero-financial-effect source semantics are implemented; no fake zero journal.

DRAFT → PENDING_APPROVAL → APPROVED → POSTED, or rejection/cancellation. Submission locks the pool and freezes exact before/after valuation, current pool version, rule selection, materialized lines and policy snapshot. Creator may refresh an unposted valuation by incrementing the source revision, discarding approval authority (not history). Changed quantity/value requires cancellation and a new attributed draft in this first immutable-line workflow. Concurrent stock changes make submission stale; posting fails closed and requires a fresh valuation and approval.

Reuse the central workflow with configurable published STOCK_GAIN/STOCK_LOSS threshold policies; absent policy uses company segregation flags and inventory:approve, not journals:approve. Require explicit inventory:view/create/submit/approve/post/cancel and inventory:cost:view; all valued workflow reads/writes require cost visibility. No role-name authorization. Initial demo grants are explicit; no operational balances/rules are seeded.

DB contracts: normalized immutable adjustment line; immutable versioned submitted valuation; existing source/approval/journal/movement/projection tables reused. Single-pool advisory locks serialize absent and existing balances. Movement insert must match the approved current valuation, same actor/source/revision/date/warehouse/product, exact projection version and original approval. Projection writes are trigger-derived, never granted as editable balances. Reject source dates earlier than the pool's latest valued date. SQL recomputes value/quantity deltas independently and binds posted stock-control lines to reviewed symbolic mappings. Complete source, rule application, movement, projection, journal, event/audit and idempotency result commit together. Deliberate later failure rolls all back.

Reuse the existing journal writer through an internal prepared-line method; the manual HTTP route remains non-control-only. Posting uses the reviewed submitted mapping, rechecks current applicability/account eligibility and freezes source/rule/valuation evidence. Retirement cannot invalidate an applied rule because journal_posting_rule_applications is populated. Gain: Dr inventory / Cr reviewed gain account; Loss: Dr reviewed loss account / Cr inventory. No tax or AR/AP effects.

Read endpoints are branch-scoped, bounded search/pagination and movement drilldown. Legacy manual-journal document routes must not expose inventory source payloads to unrelated readers. Product purchase reference price is not used. Audit costs require inventory cost visibility; general financial-ledger access still permits authorized GL amounts, not arbitrary product/stock payloads.

Full reversal implemented: a separately numbered STOCK_REVERSAL draft links the original posted gain/loss. Submission freezes the exact negation of original quantity/value and GL mapping, not current average. Original rule may be retired; current accounts must remain eligible. Original-source lock and unique active reversal prevent duplicates; cancelled requests may be replaced. No reversal chains. Native checks bind original movement/journal/rule identity and journal.reversal_of. Nonnegative resulting quantity/value and zero-quantity/zero-value consistency remain mandatory. Partial returns, transfers, zero-value stock, serial/batch/expiry, original-cost supplier/customer returns and multi-line adjustments remain later contracts. No posted rows may be edited/deleted meanwhile.

Acceptance: real reference/product/rule fixtures through governed APIs; independent three-user gain/loss workflow, exact weighted averages, stale version/reapproval, concurrent issue race, idempotency, strict actor/account/cost rejection, current permissions/warehouse/account/period checks, SQL projection/movement/GL/audit forgery rejection, deliberate rollback/same-key retry, source/stock/GL reconciliation, actual encrypted restore, then browser workflow and permission/mobile validation. Real stock workflow, restricted-cost/read-only access, GL/audit drilldown, retained policy and actual posted lines pass browser verification. Actual encrypted018/019 restores show zero stock/projection/GL/reversal/audit discrepancies. No stock UI before database/API green.

Read UI and evidence

StockAdjustments.tsx provides scoped sources/balances, immutable valuation, actual posted account debit/credit, retained approval policy/step progress, attributable commands and full reversal. GL/audit route stock sources into this UI rather than the manual journal editor. Parent summaries refresh after successful commands. docs/browser-stock-result.json records the first independent gain/loss/reversal workflow; full250-test regression includes current stock reads and customer registration. No tax ledger is calculated or posted by these adjustments; applicable statutory treatment still requires a separate approved tax workflow.

Reviewed customer registration and AR dependency contract

Status: VERIFIED for initial reviewed registration only — backend/UI, isolated reader, full21-suite browser regression and actual019 restoration pass. Customer AR remains IN_PROGRESS, with no operational AR posting released. Inventory018 remains immutable. Customer work starts with masters before sales, payment allocations or AR posting. Reuse existing customers/customer_accounts, reviewed account/tax masters, CommandService, database attribution and audit. Do not build a second journal, tax, numbering or approval engine.

Initial bounded workflow

An authenticated customer proposer submits an immutable initial registration request. PENDING → APPLIED or REJECTED by a different authenticated customer approver, regardless of role name. Current membership, permission and company eligibility are checked again inside the serialized command. A proposal/rejection has no customer publication, ledger, journal, stock or tax effect. Approval atomically publishes the exact requested customer, its reviewed AR-control binding where applicable, the decision and complete native audit. There are no editable balances or opening-balance fields.

Company-scoped unique uppercase customer number is a business-master identifier, not a financial document number. Name, optional phone/email/address/KRA PIN reference, REGISTERED/WALK_IN type, base-currency credit limit and payment terms are proposed data. Currency is resolved by the server from company configuration. PIN format validation does not verify registration with KRA. Optional governed tax category is a classification reference only; it never overrides invoice tax eligibility, rates, recoverability or statutory documents.

REGISTERED customers require an explicitly selected active postable leaf ASSET/AR control account in the same company. No seeded account code is selected implicitly. Zero credit limit means no authorized credit exposure, not unlimited credit. The master does not itself authorize a credit sale or implement exposure checks. WALK_IN is company-unique, has zero terms and zero limit, no AR-control binding, and no phone/email/address/PIN personal identity. It is a cash-sale classification, not an invented legal person. No financial effects occur during registration.

Data integrity and attribution

Normalized customer_change_requests hold proposed fields, creator/time, reason, pending/terminal state, decision author/time/reason and the generated publication identity. Existing customers gain publication link, approver/time and version. customer_accounts is used for the explicit AR binding; it contains no balance. Same-company foreign keys constrain links. Published registration and terminal decisions are immutable, including to migration-owner ordinary UPDATE/DELETE; genuine later amendments require a separate reviewed lifecycle contract and migration.

Native triggers independently enforce current proposer/reviewer authority, independent maker/reviewer, strict initial state, immutable proposal fields, governed classification/current account eligibility, exact publication identity/fields, binding and complete audit in one transaction. The company configuration advisory lock serializes publication against account/classification changes. Duplicate numbers and WALK_IN publication races fail without duplicate masters. Idempotency retry returns the original result; changed payload conflicts. Audit corruption or any late error rolls back request/publication/binding/idempotency result together.

All financial numeric values remain canonical decimal strings in public projections and SQL-assembled snapshots. No JavaScript Number conversion. Authenticated actor is resolved by the server; no request actor, approvedBy, balance, currency, version or arbitrary metadata fields are accepted. Original proposer and independent publisher remain distinct.

Permissions, API, UI

Implemented initial permissions: customers:view (authorized customer profile/credit-policy metadata), customers:propose, customers:approve. No implicit accounting, posting, payment or AR reporting permission. Generic audit readers without customers:view receive redacted customer snapshots to avoid exposing contact and policy data through a side route.

Implemented GET /customers, /customers/:id, /customers/changes, /customers/accounts, /customers/tax-categories: bounded server search/pagination, company scope and governed picklists. POST /customers/changes and /customers/changes/:id/decide: strict bodies, CSRF, retry-stable idempotency and independent publication. No update/delete routes in this first contract. UI follows backend/native green: catalog, proposal and independent review, attribution, explicit zero-credit meaning, loading/error/empty/success/confirmation, keyboard/mobile/dark, permission-only navigation and no financial dashboard grant.

AR continuation (not implemented by registration)

Next contracts must define credit-exposure serialization/reservations and current credit hold/limit/terms, atomic invoicing with original frozen tax/FX and stock/COGS snapshots, customer subledger/control reconciliation, partial/full payments and allocations, advances/unallocated cash, original-source credit/debit notes and linked reversals. Statements/aging derive from immutable transactions and allocations, not editable balances. Customer profile, control-account and credit-policy changes need version checks, independent review and preserved historical source snapshots. Deactivation cannot strand unsettled operational obligations. No registered master is evidence that AR is complete.

Acceptance before release

Test-first DB/HTTP: missing routes RED; anonymous/foreign-company/RBAC/CSRF/strict-body refusal; request inertness; concurrent idempotency and number/WALK_IN races; maker denial; current account/category/currency eligibility; exact large decimal preservation; no-audit/forged-audit/native publication sabotage rollback and same-key retry; immutable terminal decision/master/binding; all actor identities retained; audit redaction; no source/journal/inventory/AR effects. Then isolated reader and independent-review browser workflows, full regression, actual encrypted restoration and schema/API/status documentation. Migration019 was applied only after its isolated acceptance and full240 backend tests passed and a fresh actual encrypted018 restore passed. Migrations001–019 are now immutable.

Executed evidence

Genuine missing-route RED: docs/testing/red-evidence/customer-registration.txt. Customer DB/HTTP suite10 tests including parent passes: strict fields/RBAC/company/CSRF, inert/idempotent requests, true maker denial even with both grants, exact maximum credit limit, duplicate/WALK_IN races, current account eligibility, rejection, no-audit denial, audit sabotage rollback, post-BEFORE-guard publication corruption rollback/same-key retry, immutable masters/bindings and zero operational effects. Latest full250 backend regression and API build pass. Deferred native guards recheck the final rows, not merely the values seen by the first BEFORE trigger.

Workflow and isolated-reader browser receipts: docs/browser-customers-result.json, browser-customer-access-result.json; zero page errors, maker/reviewer separation, no finance/mutation authority for readers, exact amounts and keyboard/mobile/dark. Screenshots inspected. Browser inspection fixed an ambiguous credit-limit label and reusable picker URLs without an existing query string. Final optimized build/typecheck/presentation/production-web smoke and six recovery tests pass after those fixes. Actual019 encrypted restore retained one publication/binding with zero exact audit/identity or stock/GL discrepancies. Source-fingerprinted21-suite regression passed before the later pure, unreferenced credit helper; no exposed API or UI changed. Do not infer full AR completion.

Reviewed customer profile and credit-policy amendments (020)

Status: VERIFIED for this bounded metadata amendment workflow, 30 September 2026. Design preceded implementation; applied001–020 are immutable. This increment changes reviewed master metadata, not AR balances, transactions or credit-sale authority.

Bounded workflow

An authorized proposer opens a governed customer, reads its current integer version, and proposes the complete replacement editable profile: name, optional phone/email/address, credit limit, terms days and credit hold. PENDING → APPLIED/REJECTED by a different authorized reviewer. No direct PATCH of the master. Pending proposals are inert; applied/rejected requests and their exact before/after snapshots are immutable. Multiple proposals against one version are allowed; only the first approved one can apply. Stale proposals may be rejected, not rebased invisibly. To change them, create a new reviewed proposal.

Number, company, customer type, currency, tax PIN/category, active state, original publication/maker/reviewer and AR-control binding remain unchanged. WALK_IN retains zero limit/terms, no hold and no personal contact data. No account remapping, deactivation, financial FX or tax identity amendment is enabled. Zero credit limit means no credit. A hold or lower limit does not erase debt; operational credit enforcement is unreleased. Until AR source/commitment integration exists, credit-policy amendment fails closed if any customer_transactions or sales/order/return/payment operational reference exists; do not claim an empty-ledger sum is a complete exposure check. Profile-only edits can retain existing account metadata but never alter original financial snapshots.

Authorization, locks, attribution

Read/history requires customers:view. Propose/review requires separate customers:amend:propose/approve. Any changed limit, terms or hold additionally requires customers:credit:propose/approve at both proposal and decision time. No role-name authority; current session/membership/permissions checked inside CommandService. Independent reviewer mandatory even with both grants. The API rejects actor, balance, account, tax, currency, active and arbitrary fields. Original creator and original publisher remain unchanged; amendment creator/reviewer are retained separately in the normalized request and audit. Native guards recheck live grants and reason.

Reuse exclusive company configuration lock77 and row locks, serializing policy changes against current configuration and concurrent approvals. Future transactional credit sources must share the documented company/customer locking discipline before activation. No unlocked exposure preview becomes posting authority.

Schema/integrity

Add customer_amendment_requests with tenant/customer link, expected_version, proposed editable fields, exact SQL-captured before_snapshot, state, original proposer/time/reason and independent reviewer/time/reason. Partial unique index on applied(customer,expected_version) and version+1 update prevent two approvals at one version. Keep existing customers version, relax only its version upper bound through020. Keep original row structure and initial identity; record latest amendment attribution through history, not overwriting original approved_by.

The existing initial-registration guards remain for INSERT; UPDATE of governed customers requires the exact matching APPLIED amendment in the same transaction. All noneditable fields compare unchanged; native expected after snapshot is derived from the immutable before snapshot plus proposed fields/version. BEFORE and deferred final-row guards reject unaudited updates, forged requests and post-BEFORE-trigger sabotage. Customer-account bindings remain immutable. Deferred request/row guards verify exact final master and audit; any later failure rolls back request decision, customer update, audit and idempotency result together.

Freeze initial publication semantics: customer_publication_snapshot continues to return the original published snapshot after amendments, using the first applied amendment's immutable preimage. Historical registration audit must not be rewritten or compared to mutable latest metadata. Current snapshot is a separate function. Restoration tests independently compare original registration, amendment chain, current version and retained exact audits.

Implemented API/UI

GET /customers/{id}/amendments: paginated company-scoped history with exact proposed/preimage monetary strings, actor display names and review reasons. POST /customers/{id}/amendments: strict expectedVersion and full editable replacement plus reason; server captures current preimage. POST /customers/amendments/{id}/decide: independent APPROVE/REJECT and reason; no proposed amounts accepted at decision.

After native/API green, customer detail offers Propose amendment and Amendment history. Explicit version, before/after comparisons and credit-change warning; disabled current-policy edits when credit authority absent, no balance input. Retained actor identities and exact decimals; confirmation, errors/loading/empty/success, keyboard/mobile/dark. Reuse existing request/idempotency adapter and dialog behavior.

Acceptance

Real missing-route RED, then DB/HTTP tests for strict fields, current scoped permissions, both-grants maker denial, inert proposal, exact large decimals, no-op refusal, idempotency and changed-payload conflict, concurrent version race, stale rejection, credit grant revocation, operational dependency refusal, snapshot/audit/master sabotage rollback and same-key retry, original publication preservation and no financial/stock effects. Browser independent workflow plus restricted profile/credit controls; full regressions and actual020 encrypted isolated restore before bounded verification. Never edit an applied migration.

Executed evidence

261 backend tests pass (amendment suite11 including parent): granular current permissions, CSRF/scope/strict fields, optimistic races, independent review, retained exact original/audit, unsupported operational references, WALK_IN anonymity/zero-credit and deliberate late request/master/audit sabotage with rollback/retry. Genuine API and browser RED retained. All23 browser suites pass on frozen020 baseline2026-09-30T06:04:06.478Z; read-only/profile-only/workflow plus loaded mobile-dark screenshot inspected. Optimized web build/typecheck/presentation/production-web smoke and generic recovery tests pass. Actual encrypted020 restore2026-09-30T06:05:07.619Z verifies original publication, contiguous applied chain, current policy/version and exact original proposal plus approve/reject audit, with zero checked discrepancies. Evidence: ../test-results.txt, ../testing/browser-regression.json, ../testing/customer-amendment-restoration.json, ../deployment/customer-amendment-build-results.txt. No AR or production completion claim.

First AR source: reviewed service credit invoice — preimplementation contract

Status: IN_PROGRESS (design only). No invoice API, AR posting or tax register is released by this document. Preserve applied001–020. The first AR source must represent an actual business event, not an invented opening balance or an unlocked credit-check endpoint.

Why this source is next

Customer masters, explicit AR bindings, versioned reviewed credit policy, exact credit/date arithmetic and the central tax/rule/journal engines now exist. A bounded single SERVICE-product credit invoice provides a coherent first receivable/revenue/tax transaction without pretending stock was sold. Restrict to governed active SERVICE products; no STOCK or NON_STOCK ambiguity, inventory movement, COGS, cashier session or payment tender. Sales of goods and mixed/cash payments remain later integrated contracts. Reuse source_documents, existing sales/sale_lines where their semantics fit, customer_transactions, tax_transactions and journal lines. Do not create another AR or tax balance table.

Source workflow and authority

Creator proposes assigned branch, eligible customer, SERVICE product/base-unit quantity, business date and reason. The server resolves base currency, reviewed AR binding, current customer policy/version, commercial price/profile and effective tax schedules. No account, actor, exchange rate, unit price, tax factor, debt/exposure, paid amount or override flag is accepted from the frontend in this bounded source. Discount/price override requires a separate reviewed policy and is not silently enabled.

DRAFT → submitted immutable valuation/tax/credit snapshot → policy-driven independent review → APPROVED → POSTED. Follow the existing CommandService, workflow snapshots/quorum, allocator and prepared journal writer. Stale product price/tax/rule/customer policy/credit exposure requires refresh and reapproval; never silently post a different reviewed total. Posting must re-read authoritative eligibility and credit under locks even if an informational preview was recently approved. A rejected/cancelled pending source has no financial effect.

Due date is business date plus retained calendar-day terms using customerDueDate; immutable invoice customer identity, terms and policy revision persist across later profile changes. Zero limit, hold, inactive or WALK_IN customers fail credit eligibility. No fiscal compliance is inferred from account posting or a syntactically valid KRA PIN.

Exact amounts and accounting

Use the existing exact tax calculator with server-resolved effective product tax profile and governed schedules; freeze its reproducible complete output. Functional-currency-only invoice; no foreign-currency FX fiction. Positive gross only; zero-cent source posting and compounded taxes remain unsupported until separately defined. Quantity respects governed base-unit precision; pure-service invoices have no stock movement/COGS and must explicitly assert that absence.

Introduce a sealed posting event contract through a new migration. Proposed symbolic legs: AR debit for gross; revenue credit for net; output-tax control credits for exact classified tax components. Zero components must not create fake zero journal lines. Tax-exempt/zero-rated/non-taxable remain distinct immutable classifications, not a hardcoded 0% shortcut. Extend/reuse the materializer only with test-first evidence if its current contract cannot express optional/multiple tax legs; do not bypass it with unchecked client lines.

The rule-selected AR account must match the customer's independently reviewed binding. No fallback to a hardcoded account number or arbitrary customer-specific override. Until customer-selector rules are released, incompatible branch/company mappings fail visibly. Tax control mappings must match the authoritative tax definition/profile and current account eligibility. Actual journal lines, customer subledger and tax ledger must carry the same source/company/branch/currency and exact signed amounts. Native constraints independently validate these links and reject balanced-but-wrong journals.

Credit exposure and locking

Lock the company configuration scope using the common coordinator, then the source and company/customer credit scope in a documented consistent order. Every future writer that changes credit availability must participate. Financial periods and account eligibility are checked in the same transaction. No source is authorized merely by creator approval or frontend-calculated headroom.

For the first bounded invoice/full-reversal source set, derive debt from the actual released invoice/reversal customer subledger and assert that no unknown customer financial source or active commitment is omitted. Commitments, arbitrary payments/credits and order conversion are not released yet. Do not treat the absence of a new reservations table as proof of zero exposure if older/unreleased operational references exist. Fail closed for those dependencies.

Use `evaluateCustomerCredit` only after authoritative amounts/policy are resolved and locked. Its output is an arithmetic snapshot, not authorization. Compare reviewed credit state against current source-compatible debt; two simultaneous invoices cannot both spend the same headroom. Posting creates the receivable, so it may not also leave a duplicate commitment. If reservations are introduced before this source, their `source/expiry/cancellation/conversion/native` guards and reconciliation must be completed first.

020 conservatively blocks credit-policy edits for operationally referenced customers. A later migration may replace that blanket refusal only with explicit current-exposure-aware amendment rules under the same locks. Lowering a limit below debt or placing a hold must never erase debt; new credit must remain blocked. This change requires its own native race/rollback tests and visible warnings.

Atomic effects and reversal

One local transaction commits approved source state, invoice header/line/customer/tax/credit snapshots, exact balanced journal, one linked customer transaction per AR line, exact tax transactions per component, posting-rule application, attributable event/audit and idempotency result. Deliberate failure at each stage rolls everything back. No mutable customer balance, paid flag or backdoor manual AR journal.

Before calling this invoice lifecycle verified, implement a linked full original-value reversal using the original stored `net/tax/gross`, original AR/revenue/tax lines and original customer/tax identities. No current-price/current-rate recalculation. One active reversal per original; no chains. Check current eligibility and periods. When allocations/payments later exist, the original reversal must not strand them; block until the original-linked allocation/refund contract can unwind them atomically. A reversal is not a generic customer credit unrelated to an invoice.

Fiscal/external boundary

An internal accounted invoice is not evidence of eTIMS acceptance. Keep fiscal state separate and explicit (`unconfigured/pending/accepted/rejected` where implemented). No fake eTIMS identifier, receipt, callback or successful provider response. Activation requires the current contracted provider specification, credentials and accountant approval. Development source fixtures must remain visibly DEMO/non-fiscal. If legal fiscal issuance timing requires blocking a production action, implement that gate rather than silently printing a compliant-looking invoice. External calls stay outside the local DB transaction with an attributable idempotent outbox/adaptor; do not fabricate production success.

Read UI and reconciliation after backend green

Separate AR/source permissions, financial visibility and customer-profile access; PII and credit snapshots must not leak through generic audit or legacy manual-document routes. Bounded server pickers/search/pagination, immutable detailed snapshots, separate `maker/reviewer/poster/reverser`, actual AR/GL/tax drilldown, `errors/loading/empty/confirmation` and `keyboard/mobile/dark`.

Single-snapshot AR control versus customer subledger reconciliation with visible missing/mismatched source/customer/amount links; separate tax-control/ledger reconciliation and whole-journal arithmetic. Existing inventory reconciliation must remain unchanged and still pass. A non-stock source must assert no inventory effects, not ignore the inventory ledger. Statements/aging and payments are not verified merely because a first invoice posts.

Acceptance before migration activation/UI

- Genuine missing-route/backend RED; strict actor/price/account/rate/exposure/paid-field rejection, current permissions/CSRF/company/branch and independent review.
- SERVICE-only governed product, quantity/date/period/profile gaps, exact price/tax/currency capacities and no inventory effects.
- Locking: two simultaneous invoices near limit; invoice versus policy/hold/rule/period changes; stale snapshots/reapproval; idempotent concurrent posting and mismatch refusal.
- Native forged subledger/customer/control/journal/tax/snapshot/audit, including post-BEFORE-trigger mutation, rollback and same-key retry.
- Original-cost/value full invoice reversal, duplicate/reversal-chain refusal and original identity retained after later master changes.
- Meaningful nonempty AR/GL/tax reconciliation and injected discrepancies, no hidden adjustment. Actual encrypted restoration of all source/subledger/tax/journal/audit/reversal chains.
- Independent operational browser/access/mobile workflow, full regressions/build/docs; no parent AR/POS/tax or production completion claim until their remaining flows and full reconciled scenario pass.

Repository inspection and concrete integration choices (30 September)

- `AccountingService.postPrepared` remains the existing sole prepared journal writer (period locking, numbering, balanced lines, posting, linked reversals and outbox). Establish native domain authority before invoking it. No manual control-account bypass.
- `TaxConfigurationService.calculateConfigured(c, ctx, body)` already resolves governed effective schedules in the caller transaction. `ProductConfigurationService.preview` resolves dated `product_tax_profiles/product_taxes`, immutable selling price and price mode but currently owns its transaction. Extract/reuse a caller-transaction resolver rather than calling the HTTP preview or duplicating its calculator. Invoice input accepts neither tax IDs/factors nor preview output.
- `materializePostingLines` already omits zero-valued legs after validating complete mappings and amount keys; no new optional-leg engine is needed. The first source can use one sealed AR/revenue/output-tax three-leg contract. All effective rate output accounts must match its output-tax leg. Different output accounts fail explicitly until a multi-control contract is designed. Exempt/non-taxable sources still validate the complete configured mapping but create no zero tax journal line.
- Reuse `sales/sale_lines`: CREDIT, no terminal/cashier/order, SERVICE product, zero COGS; assigned active sellable warehouse is branch/configuration scope only, not a stock movement. Derive totals/due date; normalized immutable revision snapshots, not editable JSON posting authority.
- Reuse `customer_transactions` with its unique journal-line link; tighten native customer/source/account/amount completeness and immutability, not a parallel ledger.
- `tax_transactions` currently requires `rate_id/rate_snapshot/account_id`. Rated/zero-rated components have actual reviewed references; zero tax has no fake journal line. Exempt/non-taxable classification/base must remain in source snapshots and reports; never invent a zero-rate ID. Decide/test a normalized base register or constrained nullable fields before claiming the complete register. This schema decision remains open.
- Shared company lock77 already serializes commands against exclusive master changes. Add documented customer credit locking before deriving debt. Unknown operational references fail closed; explicitly distinguish inert drafts from commitments and do not double-count the current invoice.

Acceptance API target `/customer-invoices,/:id/submit|approve|reject|post|refresh|cancel,/:id/reversal-drafts`; body `warehouseId/customerId/productId/quantity/date/reason`. Separate `customer-invoices:view/create/submit/approve/post/cancel/reverse` and branch scope. **Not released routes/grants yet**. Fixtures use synthetic prices/taxes, not statutory defaults.

Deployment and activation gates

Not a production-complete ERP. Local Next production build/HTTP/CSP checks passed (`next-production-result.json`). Compose YAML/private-port/auth-policy checks passed (`config-verification.json`). Docker image builds, container restart/recreation, Ubuntu installation, and a live Cloudflare tunnel have **not** run: Docker CLI 29.8.1 and Compose 5.5.1 were installed for diagnostics. User namespaces work, but the actual RootlessKit/slrp4netns daemon attempt failed creating its TAP interface (open: Permission denied); the official launcher also rejects the host-network driver. Native docker `compose config --quiet` passed for all three manifests. No working daemon/image/container runtime was obtained. Run those gates on a Docker-enabled staging host before promotion. An API readiness response is not production certification.

Ubuntu installation and release layout

Use a reviewed supported Ubuntu release with Docker Compose v2 packages. Review then run `sudo bash scripts/ubuntu/install.sh`. Install the reviewed source release at `/opt/erp` (the repository root, not an extra nested directory). The installer enables Docker on boot but does not grant root-equivalent Docker-group access to application users or provision production/demo credentials.

Copy `.env.example` to `/opt/erp/.env`, mode 0600. Generate **independent** values with `openssl rand -hex 32` for `POSTGRES_PASSWORD`, `RUNTIME_DB_PASSWORD`, `AUTH_WORKER_DB_PASSWORD`, `RESET_ENCRYPTION_KEY` and `BACKUP_ENCRYPTION_KEY`. Database password examples require URL-safe values; hex is suitable. Store recovery keys in a separate secret manager/offline escrow, not in the backup volume. Set `PUBLIC_WEB_ORIGIN` to the trusted HTTPS hostname, `SMTP_URL` (URL-encode credentials) and `SMTP_FROM`. Never seed a production tenant with demo identities.

From `/opt/erp`:

```
sudo scripts/ubuntu/erpctl.sh validate
sudo scripts/ubuntu/erpctl.sh deploy
sudo scripts/ubuntu/erpctl.sh health
sudo scripts/ubuntu/erpctl.sh logs api
sudo scripts/ubuntu/erpctl.sh restart
```

`deploy` validates Compose, builds, backs up an existing running database, runs checksummed migrations under the owner connection, starts API/worker/web and checks readiness. Production Compose forces Secure strict HttpOnly cookies and disables the development memory transport. Nest also refuses that transport if enabled in production. The worker uses `erp_auth_worker`, not the API or owner role. Only loopback web diagnostics are published (127.0.0.1:8080); PostgreSQL and the worker have no host port. Database network is internal. SMTP egress is limited by placing the worker on a separate outbound network. Redis/MinIO are not added merely for appearance; no implemented module currently requires them.

The API never receives owner or backup credentials. Next's private API origin is server build configuration (`API_INTERNAL_URL=http://api:3000` in Docker), not a `NEXT_PUBLIC/browser` variable. The standalone web process is non-root and sets a fresh nonce CSP; production framing is denied. Inline styles remain allowed for the existing React layout; scripts use nonce/strict-dynamic. Base/tunnel images are now pinned to registry manifest digests recorded in `image-lock.json`; response content hashes were checked. This is not an image signature or vulnerability scan. Review/update these pins through the release process, and retain immutable application release images.

Reboots, updates and rollback

Named volumes retain PostgreSQL and backup data; services use `unless-stopped`, and the Ubuntu installer enables Docker. **Those settings have been inspected, not demonstrated by rebooting this environment.** Do not run `docker compose down --volumes` on a real tenant. Test container recreation and host reboot on staging.

Set `ERP_RELEASE` to an immutable reviewed tag and retain previous images. `sudo scripts/ubuntu/erpctl.sh rollback PREVIOUS_TAG` changes application images only, never the database. Readiness compares the full migration checksum manifest, so an incompatible old application will fail closed. Do not automatically undo migrations or restore over live data: restore into a new database, verify balances and post-backup changes, and plan an approved cutover. An image rollback after a schema change requires explicit compatibility review.

Cloudflare Tunnel

Create a tunnel in **your own Cloudflare account**, route the chosen public hostname to `http://web:5173`, and store its token as private `TUNNEL_TOKEN`. Use `sudo scripts/ubuntu/erpctl.sh tunnel` after web/API are healthy. Cloudflared reaches web only via the application network and connects outward to Cloudflare; do not publish database, worker or storage services. Configure Cloudflare HTTPS and an appropriate access policy. Keep origin ports firewalled; the loopback diagnostic port is not Internet-facing. Never reuse or expose Arena preview traffic tokens.

Live DNS/TLS/tunnel behavior, provider credentials, trusted ingress IP attribution/rate limiting, external SMTP, image provenance scanning and disaster-recovery drills on the production host remain activation gates. The current API deliberately does not trust arbitrary forwarded-IP headers; review the exact tunnel/Next/Nest boundary rather than enabling a blanket trust-proxy setting.

Health semantics

- `/api/health/live`: process can serve requests; independent of DB.
- `/api/health` and `/api/health/ready`: DB query, exact migration checksums, required worker heartbeat, optional actual storage write/delete probe. Failures return 503 without credentials/SQL/paths.
- `REQUIRE_AUTH_WORKER=true`: heartbeat must be no older than 60 seconds. It proves polling liveness, **not SMTP acceptance**.
- `REQUIRE_FILE_STORAGE=true`: `FILE_STORAGE_ROOT` must exist and be writable. This does not implement business attachment ingestion.
- Redis/storage disabled checks report `not_required`, never fake successful integration.
- `productionReady: false` remains explicit until the complete ERP release gates pass.

Public health contains no credentials or exception stack. `LOG_HTTP_REQUESTS=true` emits structured request ID, route template, method, status, duration and authenticated actor ID; it deliberately excludes request bodies, cookies, session headers, query strings and passwords. Configure protected log shipping and retention before production.

Backup, restore and disaster recovery

Verified scope

restore-verification.json and test-results.txt record an actual encrypted PostgreSQL backup restored into a new database, consistent snapshot-count checks, journal arithmetic verification, restored runtime privileges/posted-write denial, encrypted file restoration, overwrite protection, corrupt/wrong-key denial, and link/traversal rejection. The original database was not replaced. This is a **local recovery drill**, not proof of production-host recovery, external offsite durability, or future inventory/AR/AP/tax reconciliation.

Native commands

Requirements: Node 22, installed repository dependencies, matching PostgreSQL 17 pg_dump/pg_restore, Python 3. Operator-only connection: BACKUP_DATABASE_URL or MIGRATION_DATABASE_URL. API credentials must not be promoted to owner credentials. Set a separate 64-hex-character BACKUP_ENCRYPTION_KEY; it is mandatory when NODE_ENV=production. Development plaintext backups are explicitly marked encrypted:false and are not suitable for offsite production storage.

```
scripts/backup-db.sh /secure/backups/database.erpbackup
scripts/restore-db.sh /secure/backups/database.erpbackup erp_restore_drill --confirm-restore
FILE_STORAGE_ROOT=/var/lib/erp/files scripts/backup-files.sh /secure/backups/files.erpbackup
scripts/restore-files.sh /secure/backups/files.erpbackup /var/lib/erp/new-restored-files --confirm-restore
```

Every archive has a sidecar .manifest.json with ciphertext checksum/size and snapshot counts for DB archives. Keep archive and manifest together. Database dump uses an exported repeatable-read snapshot; counts are from that same snapshot. Encryption is AES-256-GCM with fresh nonce and authentication tag. Key material is not included. Files/directories are private (0600/0700). Passwords are passed to native clients through environment, not command arguments/logs. Staging plaintext is removed in finally; interrupted OS processes can leave protected temporary directories, so include temporary-directory inspection in recovery operations.

Restore requires an explicit confirmation flag. Database targets must be **new** names beginning erp_restore_; live erp and existing databases are refused. PostgreSQL restores in one transaction and preserves runtime ACLs, with known non-superuser roles provisioned on a fresh recovery host. Restoration explicitly invalidates the copied worker heartbeat so a recent backup cannot falsely claim that a worker is attached to the recovery database. A new heartbeat is required. Provision fresh runtime/worker passwords separately; role passwords are not part of pg_dump. Only restore trusted authenticated archives: PostgreSQL dumps contain executable schema definitions. A failed newly created target is retained for operator inspection; no existing database is deleted. File restore likewise requires a new directory and rejects absolute/traversal paths, duplicate names, links, special entries and size-limit violations before extracting. No existing files are overwritten.

Compose operation

The ops tools profile uses PostgreSQL 17 clients, Node and Python without publishing a port:

```
sudo scripts/ubuntu/erpctl.sh backup
sudo scripts/ubuntu/erpctl.sh backup-files
sudo scripts/ubuntu/erpctl.sh restore /backups/database-ARCHIVE.erpbackup erp_restore_drill
# File restoration, with a new destination inside the restore volume:
sudo docker compose --env-file .env -f docker-compose.prod.yml run --rm ops restore-files /backups/FILES.erpbackup /restore/drill --confirm-restore
```

The production Compose file's files volume is provisioned for future attachment storage; no business attachment module is claimed. Consistent DB+files recovery requires pausing attachment writes or introducing a coordinated immutable-object snapshot/manifest before that module is enabled.

Frequency and retention policy to activate

Recommended initial policy, subject to business-approved RPO/RTO: encrypted nightly logical backups, pre-migration backups, 7 daily + 4 weekly + 12 monthly restore points, and encrypted off-host copies. This is a policy recommendation, **not a running schedule**. Configure the backup scheduler/offsite destination and storage lifecycle explicitly; no automatic deletion is performed by these scripts. Never delete the only restore-tested copy. For a low RPO, add continuous WAL archiving/PITR and test it; logical daily dumps alone cannot meet minute-level recovery requirements.

Monthly restoration drills should record timings, archive IDs/checksums, key recovery, schema manifest, counts, foreign keys, journal balance, and every implemented subledger-to-control reconciliation. Current operational subledgers are unfinished and their reconciliation remains NOT_STARTED. Rotate backup keys only with an inventory of archives encrypted under each key; retain old keys until their final backups expire under approved policy.

Disaster recovery cutover

1. Stop new business writes and preserve surviving logs/data; establish incident ownership.
2. Provision an isolated recovery host with the reviewed release and matching PostgreSQL major version.
3. Retrieve archive, manifest and independent key; verify integrity and restore to a new target.
4. Verify migration manifest, row counts, journal arithmetic, runtime privileges and every released reconciliation. Investigate differences; never silently adjust balances.
5. Reconcile transactions after the backup/PITR target. Preserve payment provider references and idempotency history to avoid duplicate external posting.
6. Provision runtime/worker credentials, start worker against the restored DB and require a fresh heartbeat; verify controlled authentication and read-only financial reports.
7. Approve and perform the connection/DNS cutover, then reopen writes and monitor. Retain the original database until incident review authorizes disposal.

Re-run the local drill with `node --test scripts/ops-test.cjs`. It creates and drops only its uniquely named isolated restore target and does not fabricate operational reconciliation success.

September29 recovery evidence

Read sandbox recovery and continuity: an older checkpoint was actually recovered, later original demo history was lost, and a fresh encrypted checkpoint is now retained. Reopening production writes requires issuance/payment/ledger continuity verification; balanced arithmetic alone is insufficient.

020 amendment-aware recovery receipt

`scripts/customer-amendment-restoration-test.cjs` performs encrypted backup and actual isolated restore, checks all migration hashes, journal arithmetic, stock/projection/GL/original-value reversals, original customer publications, contiguous applied amendment chain, current policy/version and exact original proposal plus decision audits (including rejected requests). Latest receipt: `../testing/customer-amendment-restoration.json,2026-09-30T06:05:07.619Z,62sources48journals96lines1088audits,5customers,6amendments,zero AR entries and zero checked discrepancies`. This is local recovery, not full future AR/tax restoration or production/offsite/PITR evidence. Preserve encryption keys securely outside archives. The historical initial-registration script now compares original policy through its frozen publication snapshot, not the amended current master.

Security implementation and open gates

Read the original API/security contract, phase-one controls and preview recovery contract alongside the live requirement matrix. The latter distinguishes tested core controls from unfinished operational modules.

Executed controls

- Real password verification, hashed server sessions, expiry/inactivity limits, CSRF, logout/self-revocation, password change/reset, company membership and branch scope. Login/reset responses do not disclose raw stored hashes.
 - Permissions are explicit grants; neither role names nor client-supplied actor IDs authorize commands. CommandService rechecks current user/session/membership/grants after acquiring the transaction lock, preventing stale permissions from being used after a concurrent access change.
 - Runtime PostgreSQL principal is `erp_runtime`, not the owner. Composite tenant FKs, RLS, deferred balanced-journal/source/audit constraints and immutable posted data reinforce application checks. Owner migration/recovery access remains a privileged trust boundary; do not expose its connection string to the API.
 - Auth delivery uses a separate `erp_auth_worker` principal with narrow outbox/token/heartbeat/security-event permissions. Migration 009 rejects a human actor, wrong audit action or unmatched job metadata from that role. It cannot read financial journal data. Encryption keys/provider credentials are operator secrets, never release artifacts.
 - Number allocator guards require monotonically increasing counters, immutable established formats and a matching source/journal in the allocation transaction. Financial rollback also rolls back allocation.
 - Optional JSON HTTP logs include only generated request ID, route template, method, status, duration and authenticated actor. Errors omit SQL, credentials, stack traces and payload bodies. Failed financial commands are separately audited after rollback; a failed audit persistence attempt emits an operational alert rather than silently pretending success.
-

Browser and ingress

Next production pages use a fresh nonce CSP/strict-dynamic and deny framing. Cookie sessions are Secure/HttpOnly in production and the development memory transport is forbidden at startup. Native local development can use explicitly configured non-Secure cookies; Arena's private development iframe uses explicit memory transport because cookie availability differs by browser/platform. Memory credentials are JavaScript-readable and **not equivalent to HttpOnly**; they never enter storage/URLs/logs and disappear on reload.

Do not loosen CORS, enable blanket trust-proxy, expose infrastructure ports, or reuse preview traffic tokens to solve sign-in. The API currently avoids trusting arbitrary forwarded IP headers. Exact Cloudflare → tunnel → Next → Nest client-IP attribution/rate-limit behavior must be reviewed and tested on the real ingress topology before production acceptance.

Open gates

Production SMTP delivery/monitoring; MFA and sensitive-action step-up; complete onboarding/account lifecycle; trusted ingress review; container/digest/dependency provenance and security scanning; externally exercised production authentication; business-specific permissions/approval thresholds for unreleased modules; full reconciliation/concurrency/rollback coverage across operational workflows. Current local health, unit tests and iframe simulations are not a penetration test or a production security certificate.

Private paths excluded from source releases include `.env*` except `.env.example`, `.runtime`, `.cache`, mail, backups, TLS keys, DEMO_ACCESS and generated builds. Keep per-environment credentials independent and rotate them through the documented operator path. Do not copy a development seed or its credentials into a production company.

Demo-seed production boundary

The development seed now rejects `NODE_ENV=production` before opening a database connection; a test-first regression proves zero connection attempts. This is not production onboarding and cannot protect an operator who deliberately mislabels a production target as development. Production onboarding/SMTP/step-up remain unfinished.

Delivery status — 30 September 2026

FULL ERP NOT COMPLETE. Production readiness is false. Continue autonomously. This is a resumable working checkpoint, not the final completion report. The earlier v0.5 archive is historical and does not contain current changes.

Authoritative ledgers: implementation status, requirements matrix, execution checkpoint. Earlier milestone detail is retained in historical status through017.

Current implemented scope

- Next.js/NestJS/PostgreSQL foundation; authenticated company/branch scope, granular RBAC, session/reset/revocation, attributable commands, retry-stable idempotency, native immutable balanced journals and linked reversals. Configurable financial approval snapshots retain thresholds, steps, quorums and maker/poster segregation. No role-name authorization.
- Reviewed organization, COA, document numbering, posting mappings, initial tax schedules, product references and initial SKU registration, with bounded native integrity/audit tests. Exact tax/valuation kernels are reused. Their remaining lifecycle and operational requirements are not thereby verified.
- **018 stock workflow:** reviewed single-product gains/losses and full linked original-cost reversals. Atomic source, valuation, reviewed mapping, movement, derived projection, balanced GL and audit. Locked pools, stale-value reapproval, duplicate protection and native rollback/forgery checks. No editable balances, partial returns, transfer/count/tracking or zero-cent stock posting. Separate stock and cost permissions; audit redaction and manual-document isolation.
- **Stock UI:** searchable sources and balances, governed pickers, independent review/posting, cancellation/refresh/reversal, actual journal lines, retained policy/quorum, original-source links, GL/audit drilldown, on-demand stock/projection/inventory-control reconciliation, keyboard/mobile/dark.
- **019 customer registration:** inert immutable proposal → independent APPLIED/REJECTED decision → exact customer and AR-control binding, with original maker/reviewer identities. Current account/classification eligibility, concurrent number/WALK_IN uniqueness, exact credit-policy strings, complete native audit, and deferred recheck against deliberate late row corruption. No AR transaction, opening balance or credit-sale authority is created.
- **Customer UI:** catalog and request search/pagination, AR/classification pickers, profile proposal and review, immutable detail and permission-only navigation. Generic audit snapshots are redacted without customer-profile visibility. Zero credit limit is not unlimited credit.
- **020 customer amendments:** exact optimistic versions, immutable preimage/proposal/decision, profile versus credit grants and independent review, final-row/audit native guards. Original publication/AR binding stay frozen. Operational references conservatively block credit edits; WALK_IN stays anonymous/zero-credit. Desktop/mobile history/proposal/review and least-privilege UI; no financial or stock effect.
- **Next AR-source acceptance is IN_PROGRESS, not green:** isolated governed service/tax/customer fixtures succeed, then /customer-invoices fails404 versus201 as expected. Evidence: docs/testing/red-evidence/service-credit-invoice.txt; design and future atomic posting/reversal/concurrency assertions in docs/customers/service-credit-invoice-next.md and apps/api/acceptance/service-credit-invoice.test.ts. No invoice route,021 migration or AR entry exists yet.
- Continued customer credit dependency:10 pure exact eligibility/due-date tests,3,465 cent cases, immutable replay, no endpoint/reservation/posting authority. Transactional credit enforcement and AR remain unfinished.

Executed evidence

- **261 backend tests, zero failures**,87 top-level: docs/test-results.txt; API compile passed. Stock suite15 including parent; customer registration suite10 and amendment suite11 including parents. WALK_IN and late request/master/audit sabotage included. Genuine preimplementation RED records retained.
- Stock workflow/access and customer workflow/access Chromium tests pass with zero page errors. Stock GL/audit drilldown, retained policy, exact posted lines and original-cost reversal are exercised. Customer independent publication and least-privilege read/no-mutation paths are exercised. Desktop/mobile/dark screenshots inspected.
- **All23 frozen-source browser suites PASS**, finished2026-09-30T06:04:06.478Z, respecting natural login throttle. docs/testing/browser-regression.{json,txt}; prior01921-suite receipt retained separately. Amendment workflow/profile-only/read-only checks pass with loaded mobile screenshot and viewport assertions.
- Web typecheck, optimized build, six exact-presentation tests and production-web CSP/nonce/hydration/private-proxy smoke pass. Current020 build/typecheck/unit/smoke and recovery log: docs/deployment/customer-amendment-build-results.txt. Production backend/Docker flags remain false.

- Six generic encrypted database/file recovery tests pass. Actual **020 encrypted isolated restoration** verifies all 20 hashes, original publications, contiguous amendment before/after chain, current master version/policy, exact amendment audit, stock/projection/GL and full original-cost reversals with zero checked discrepancies. No live DB overwrite or fabricated AR entries. Receipt: docs/testing/customer-amendment-restoration.json; prior 019 receipt preserved separately. Production-host/offsite/PITR recovery remains unverified.
- Exported schema: 136 tables, 1473 columns, 583 foreign keys. OpenAPI: 97 paths. Table/API counts are not module-completion evidence.

Incidents and recovery boundaries

Next development hosting can restart near its configured memory threshold; fresh stable hosts passed. Production forbids frame embedding, so it is not used as the Arena preview and CSP is not weakened.

A combined browser run hit the unchanged 60-logins/15-minute throttle. Waited for natural expiry; did not clear rate buckets. Stock tests now reuse three isolated authenticated browser sessions, and the full-suite runner pauses before exhausting the budget. The interrupted approved/unposted stock fixture was cancelled through Amina's authenticated API with a reason; all history retained. An access administrator was correctly denied stock cleanup authority.

Development fixtures are explicitly DEMO and include real test stock movements/GL and customer registrations, not real commercial activity. No provider funds or production transactions are fabricated. The earlier sandbox recovery/history-loss incident remains documented in the recovery continuity note; backups are not claimed as zero-RPO/PITR/offsite recovery.

Arena's live development preview uses tab-memory sessions, with normal expiry, revocation, CSRF and backend permissions; cookies are not required. Reload requires sign-in. Actual user-side Arena routing/login remains unconfirmed, despite local cookie-stripping/iframe tests. Credentials are unchanged and private.

Remaining modules and production gates

Further customer/account lifecycle, serialized credit exposure, AR invoicing/allocations/statements/aging; suppliers/AP; purchase requests/PO/GRN/invoice matching; POS/cashiers; unified payments and verified M-Pesa; sales/supplier returns; expenses/cash; full financial/operational reports and reconciled end-to-end scenario. Inventory still needs counts/transfers/reservations/tracking, multi-line sources and commercial integration. Tax needs operational registers, amendments and validated eTIMS integration.

Remaining security/organization/master lifecycle, MFA/step-up, external SMTP/provider contracts, meaningful load/soak/security/accessibility testing, attachments/retention/offsite/PITR and issuance-continuity policy remain visible in the matrix. Docker/Compose/operator scripts are authored and configuration checks pass, but actual rootless Docker execution is blocked here by TAP privileges. Ubuntu/Tunnel deployment, reboot/recreation/update/rollback and production-host recovery are not verified. No final 24-point completion report until practical deliverables and the complete reconciled scenario pass.